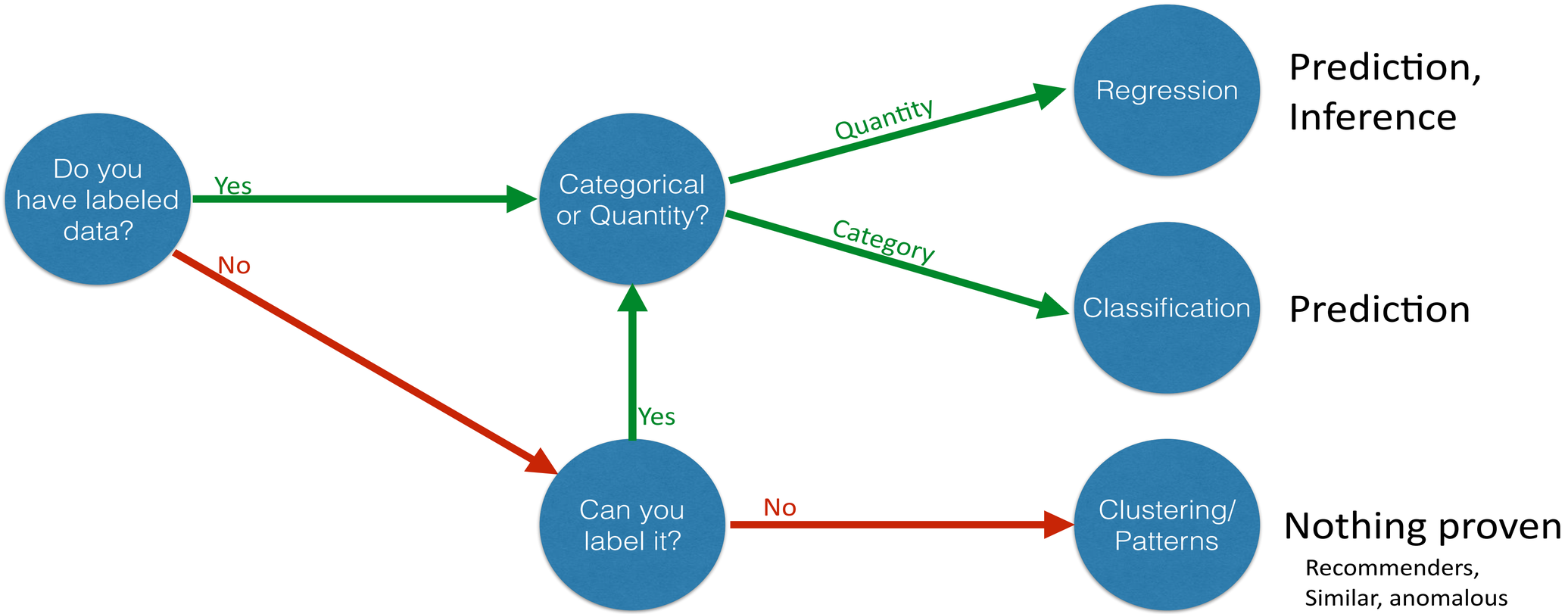
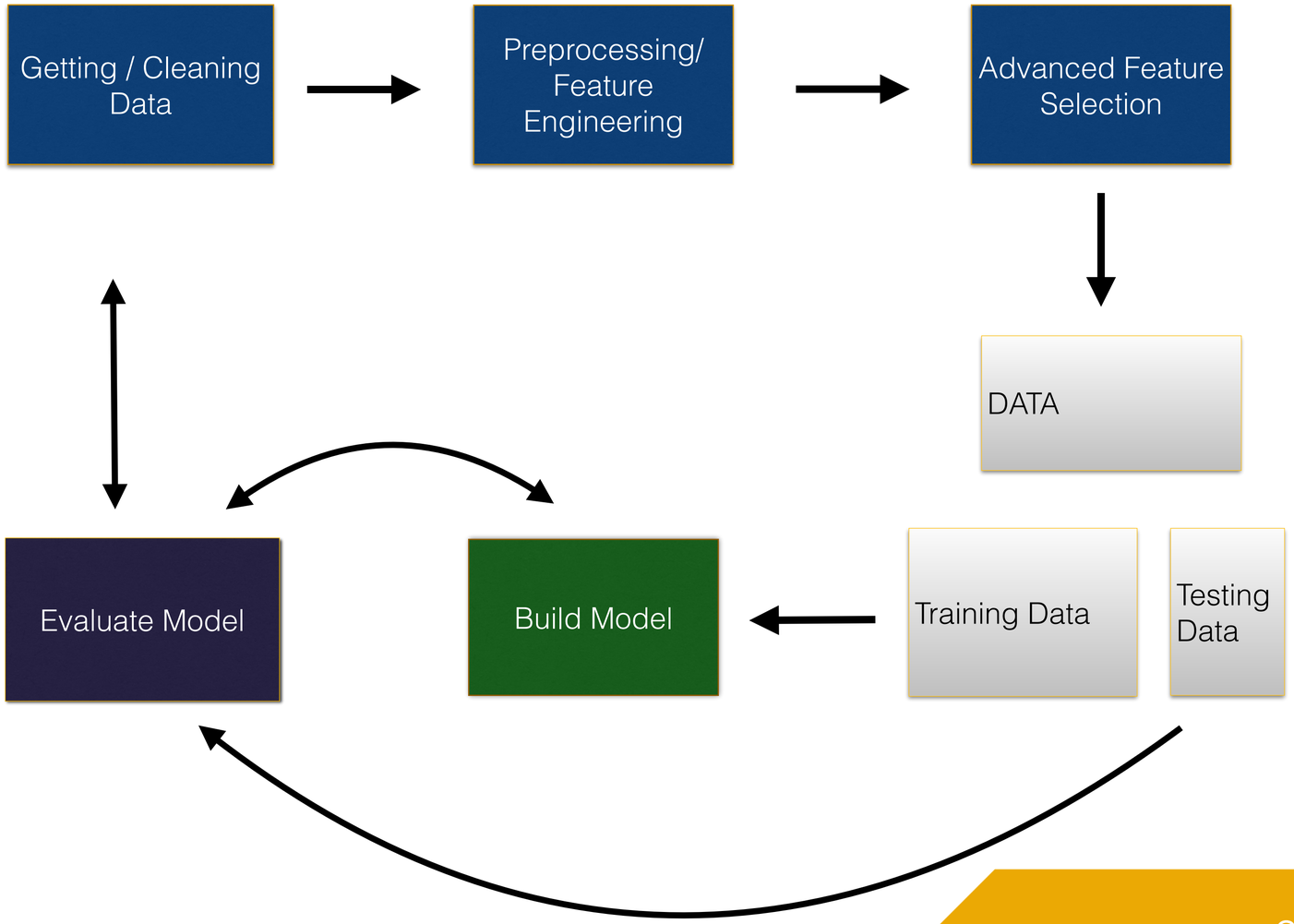
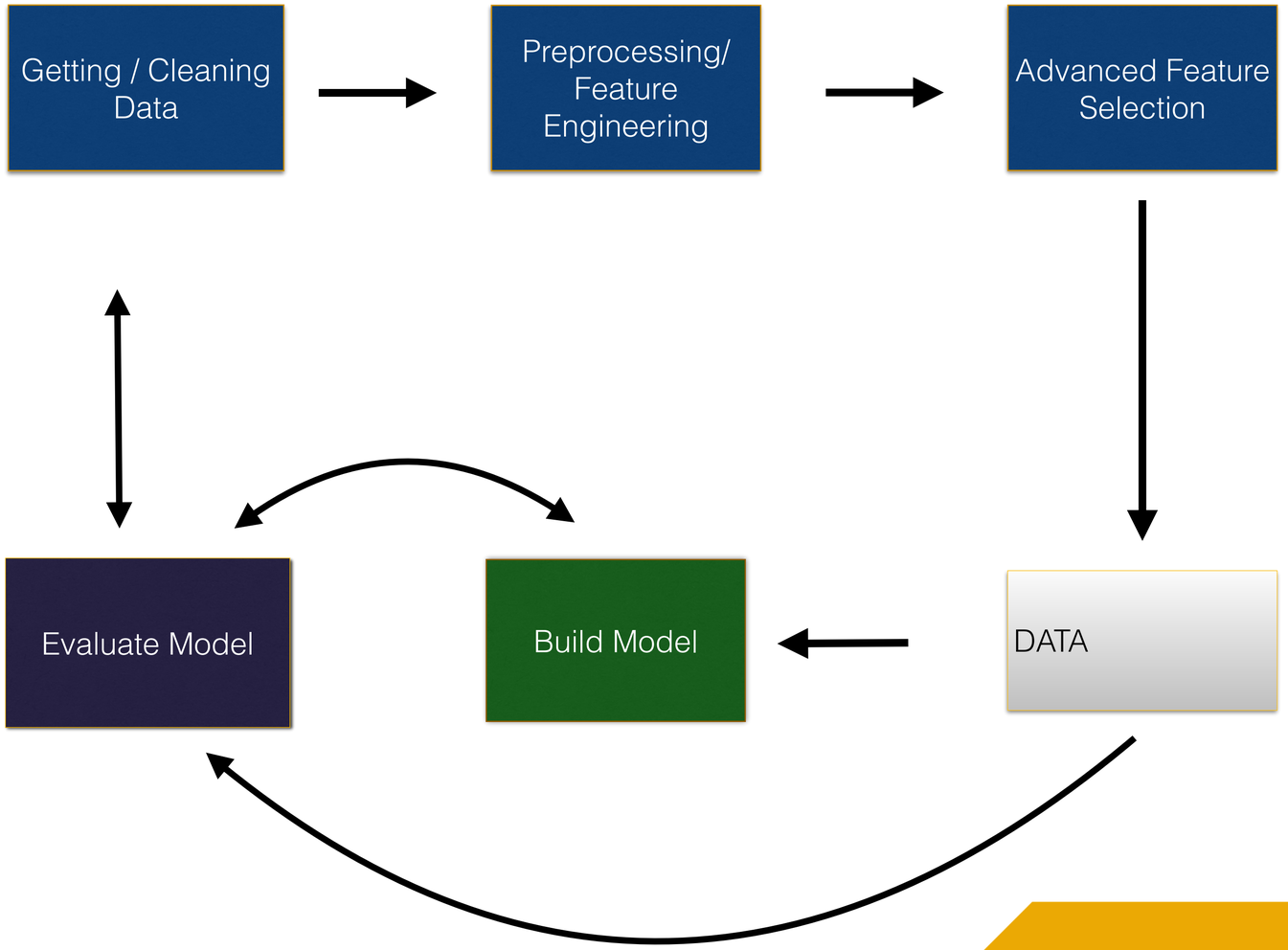
Module 5: Unsupervised Learning


Do you have labeled data?
Yes
No
Categorical or Quantity?
Quantity
Category
Regression
Classification
Yes
Can you label it?
No
Clustering/ Patterns
Prediction, Inference
Prediction
Nothing proven
Recommenders,
Similar, anomalous

Supervised Machine Learning Process



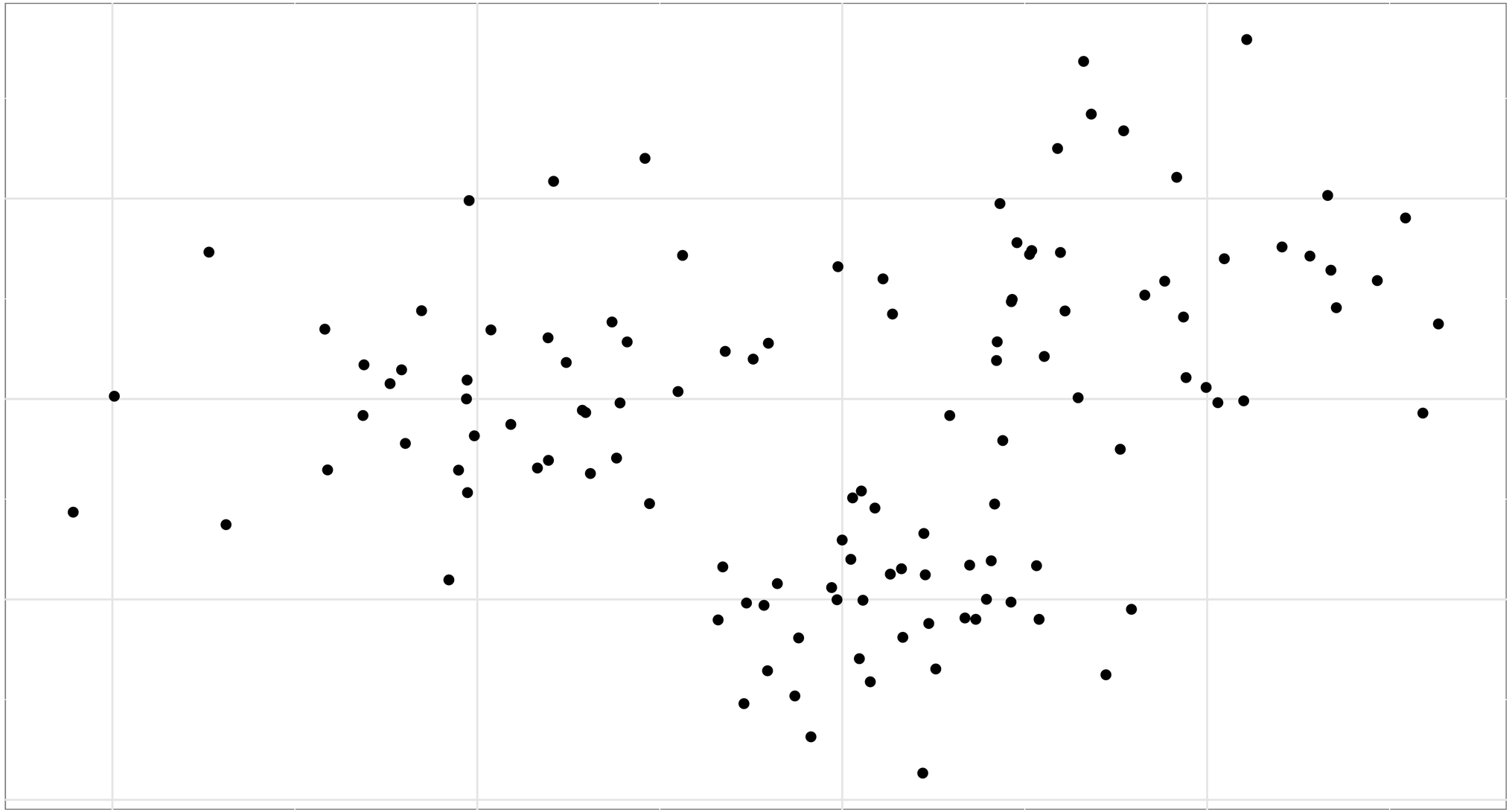
Unsupervised Machine Learning Process



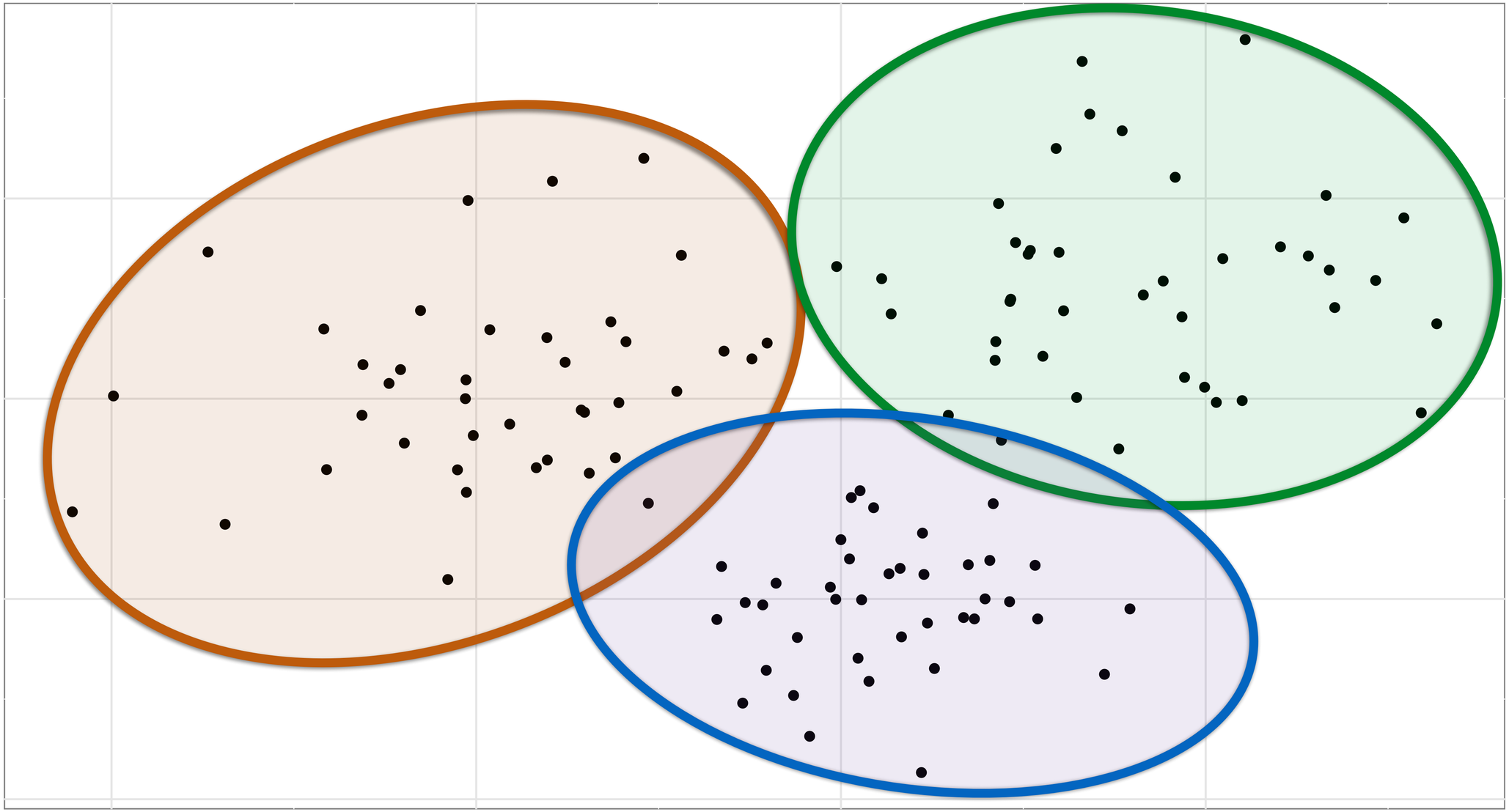
Unsupervised Clustering Algorithm

1. Select Features
2. Calculate a distance measure
3. Apply a clustering algorithm
4. Validate?

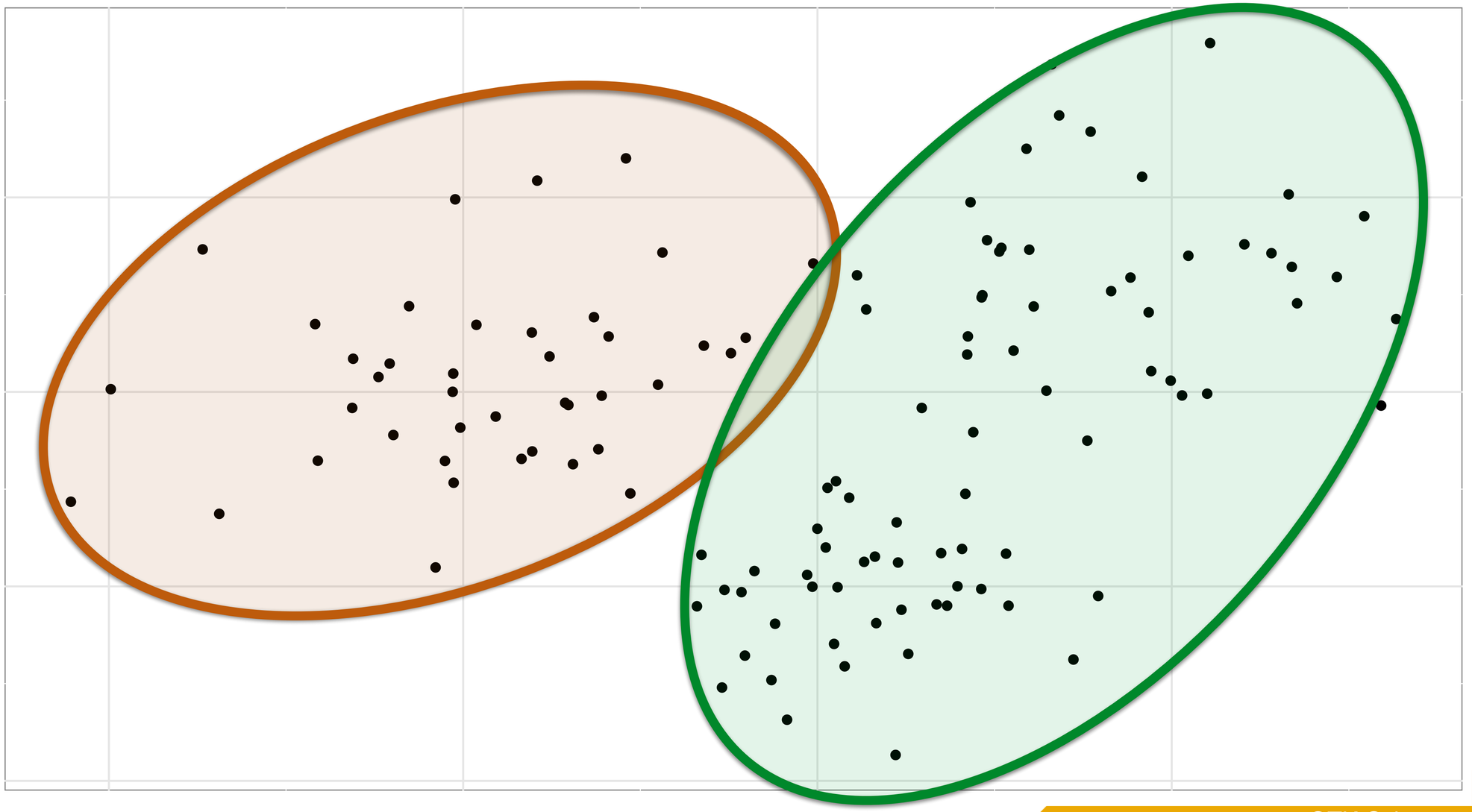
Clustering…



Clustering…



Clustering…

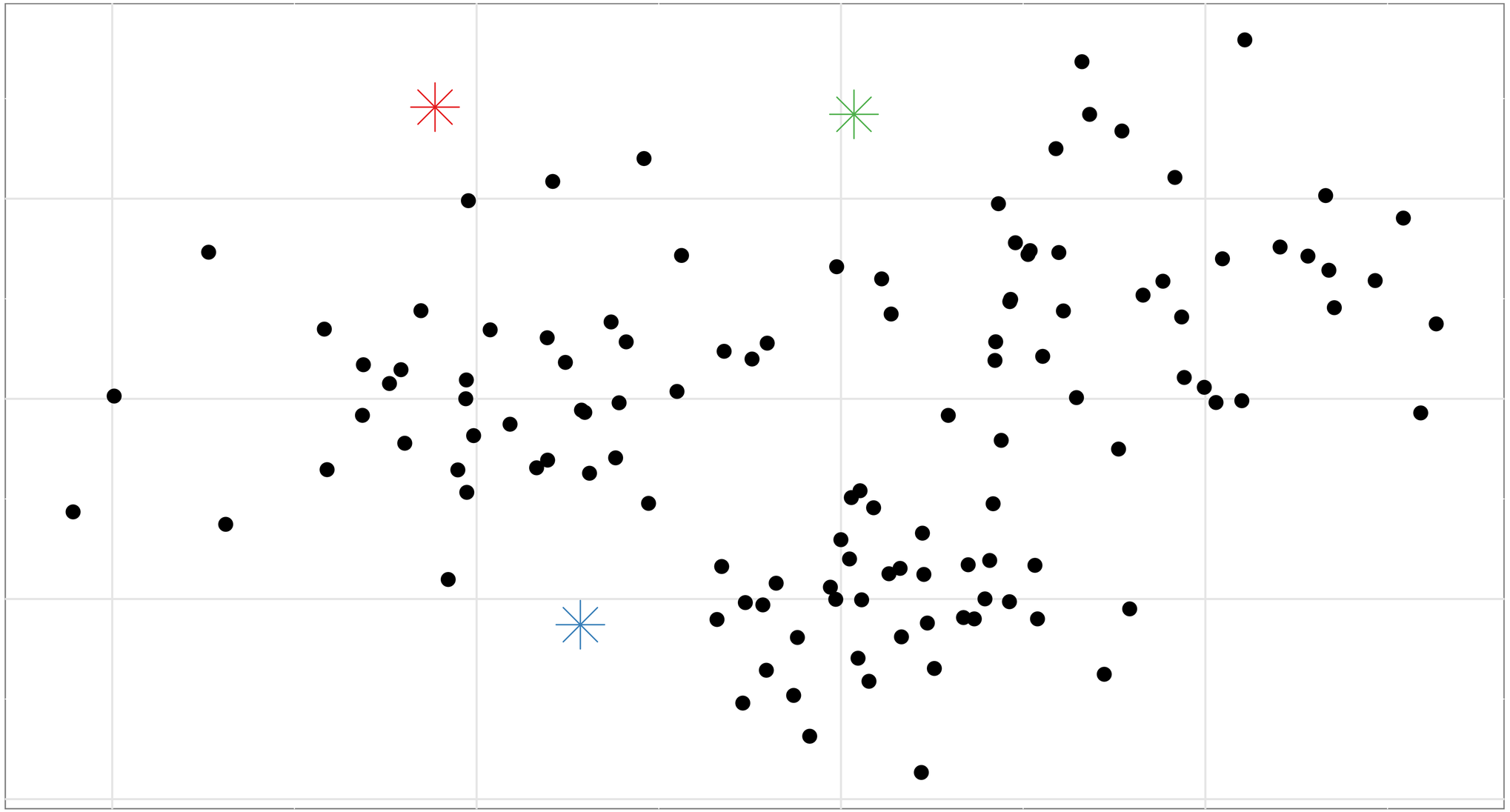


K-Means

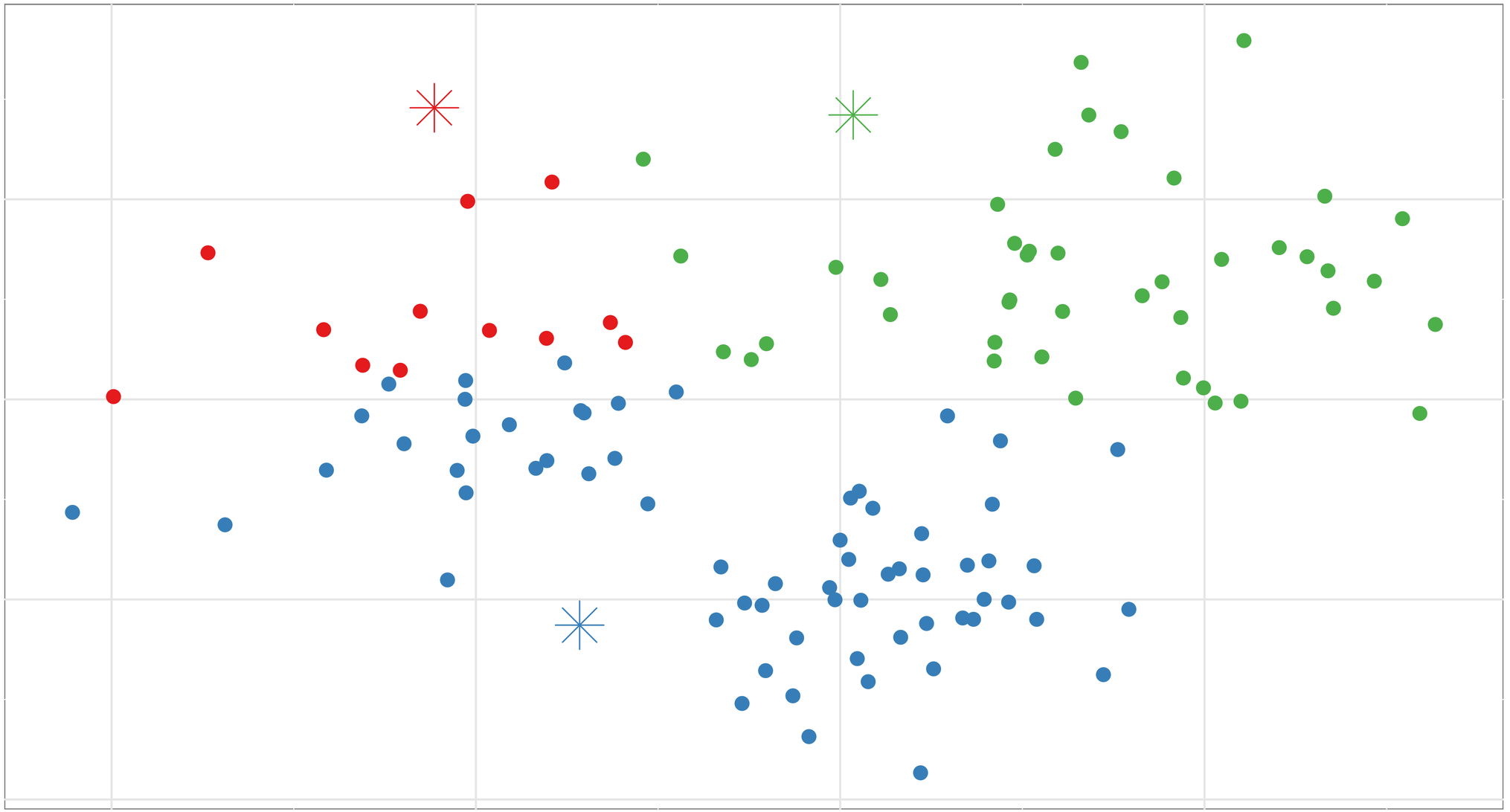
Before starting, pick the number of clusters, K

1. Pick K random centroids within data range
2. Assign each data point to the nearest centroid
3. Move centroid to center of assigned points
4. Repeat steps 2 and 3 until centroid stops shifting

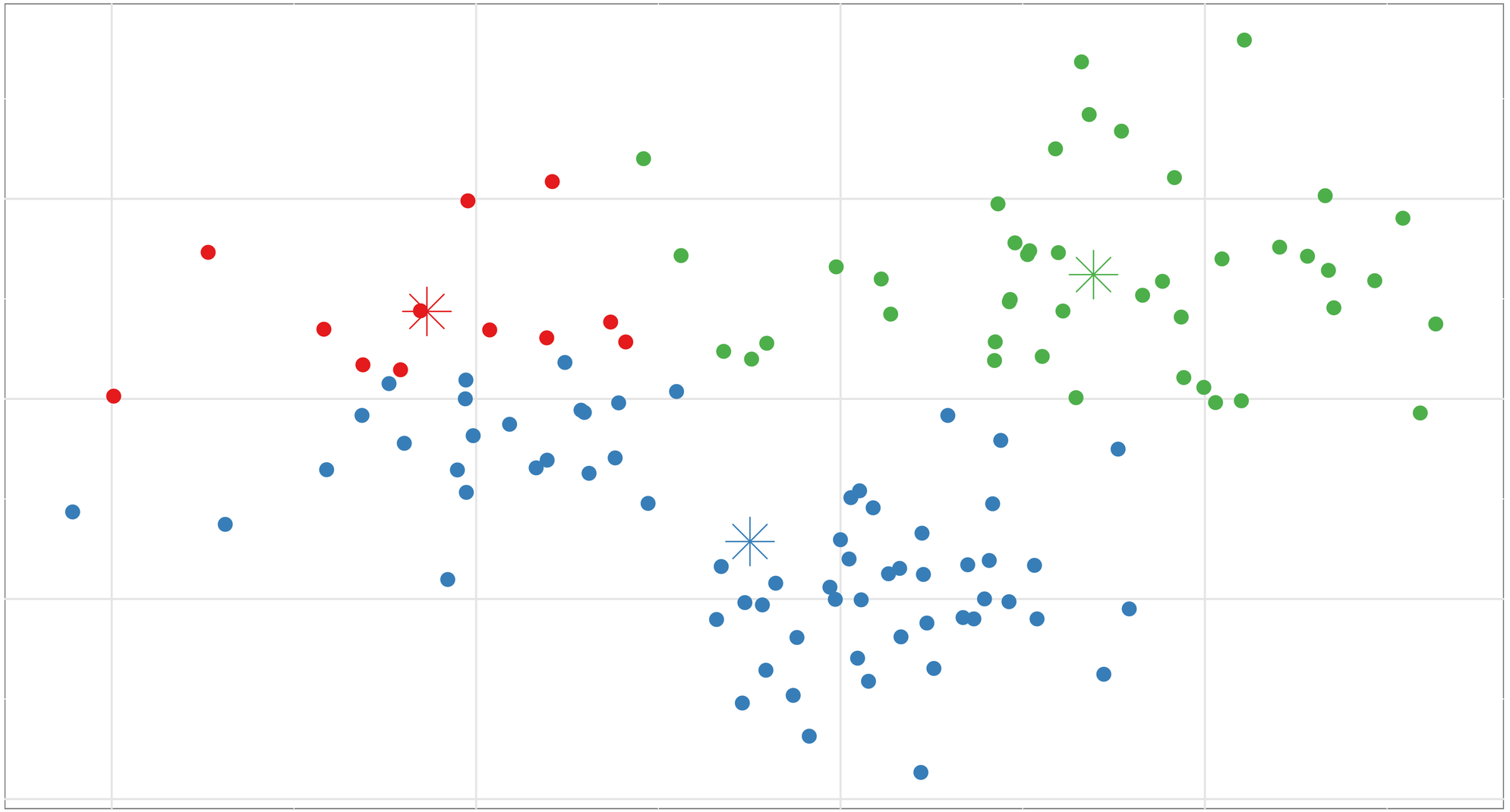
Step 1: Pick 3 random centroids within data range



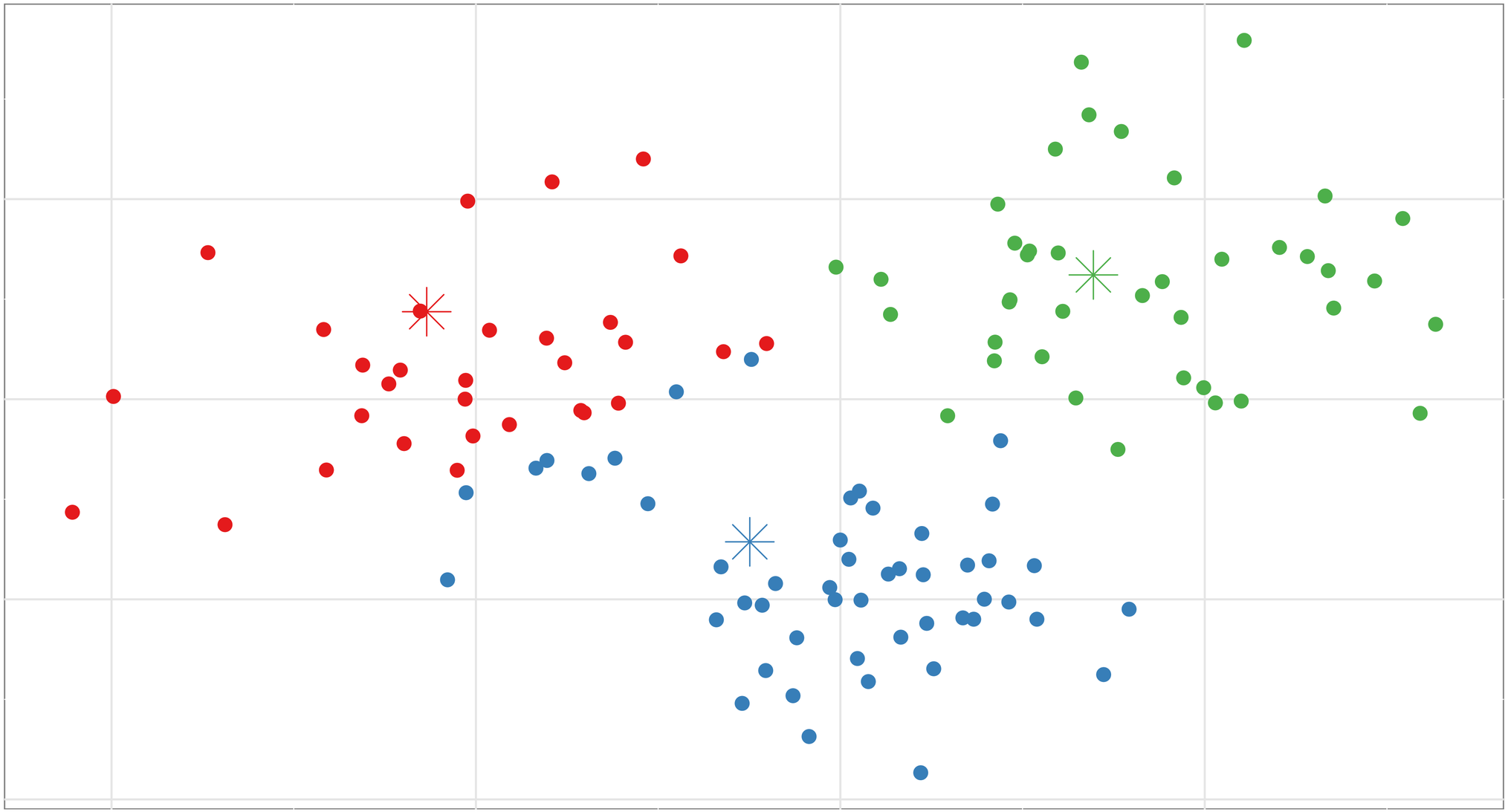
Step 2: Assign each data point to the nearest centroid (1)



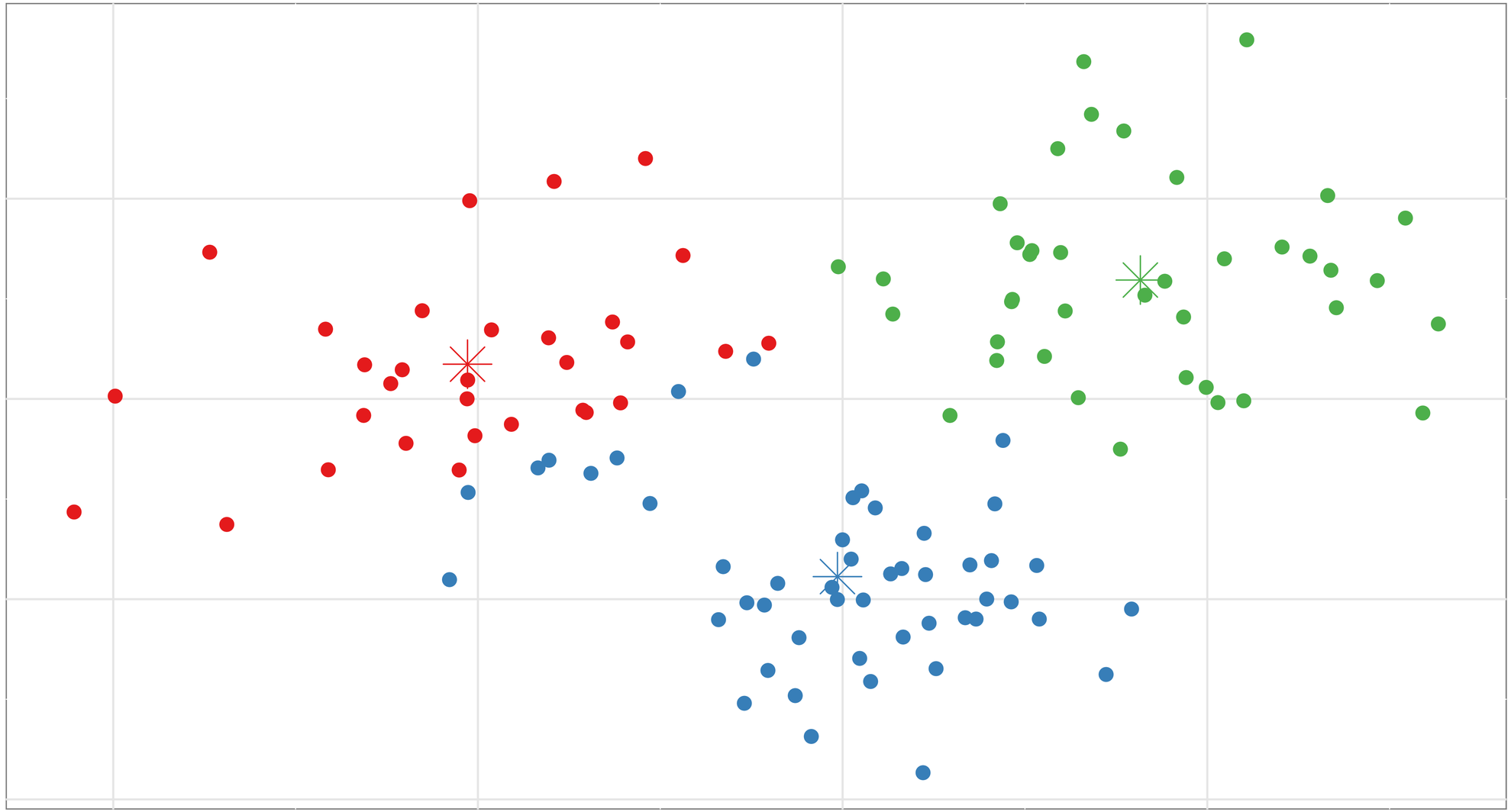
Step 3: Move centroid to center of assigned points (1)



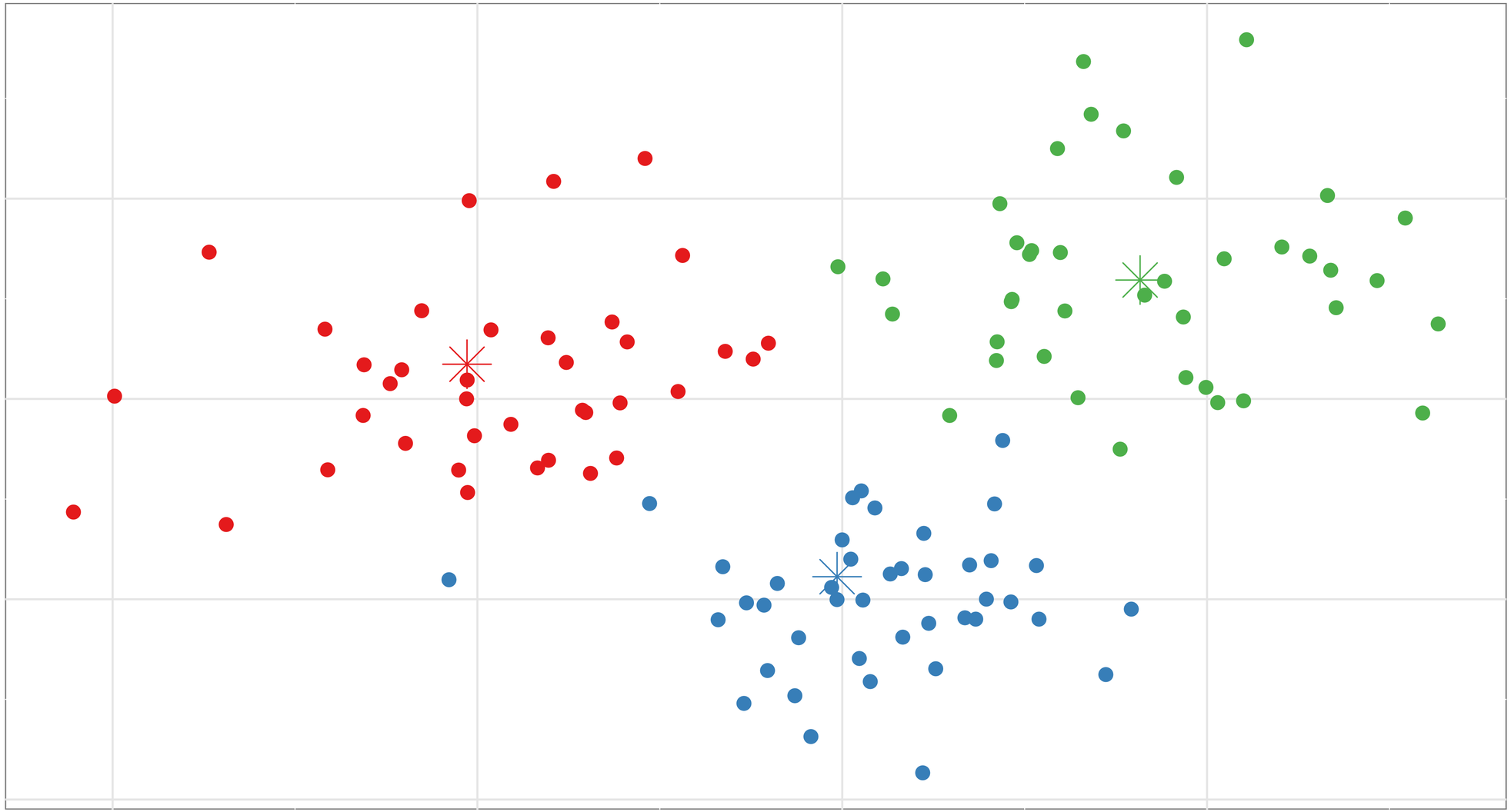
Step 2: Assign each data point to the nearest centroid (2)



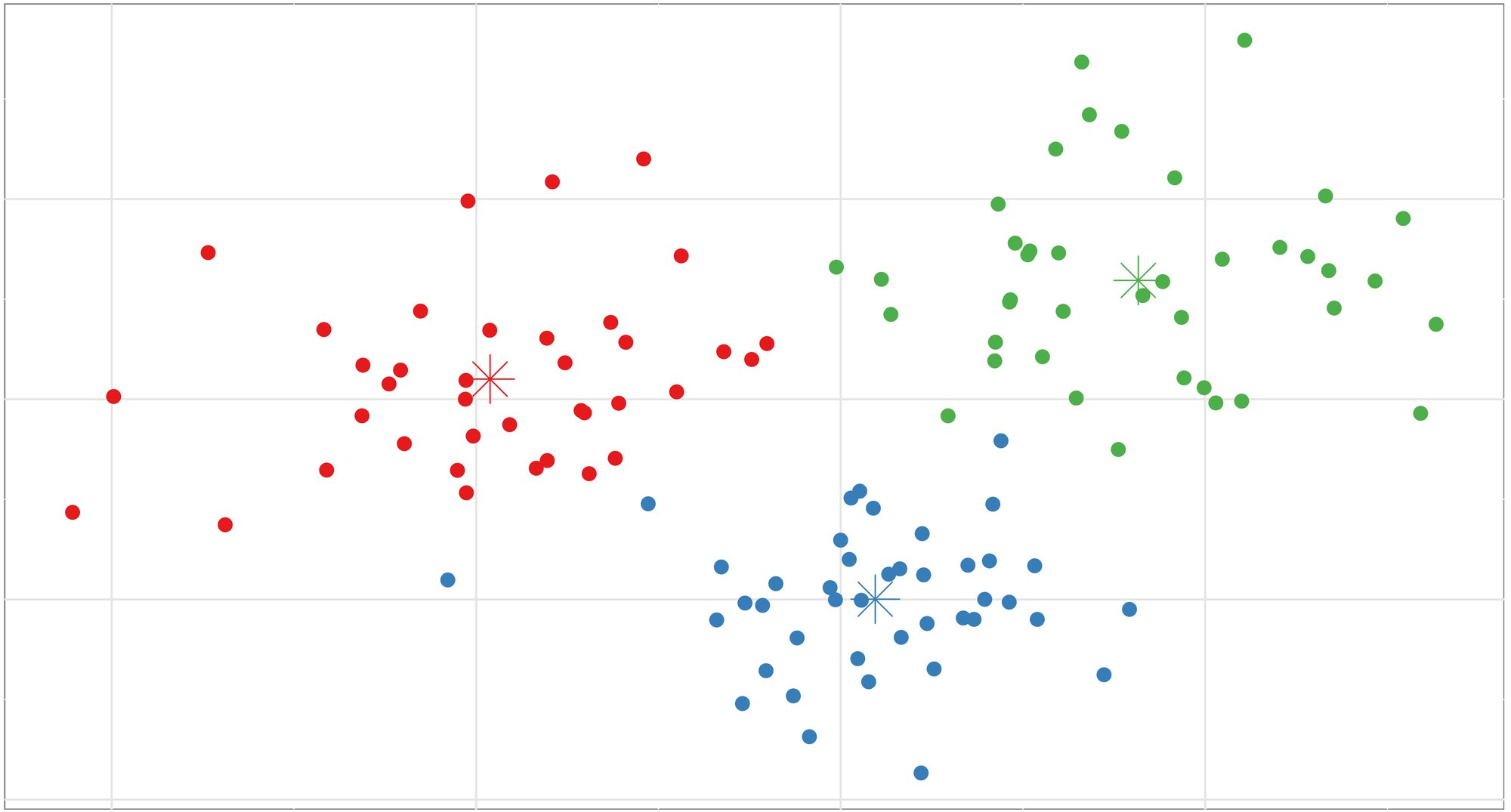
Step 3: Move centroid to center of assigned points (2)



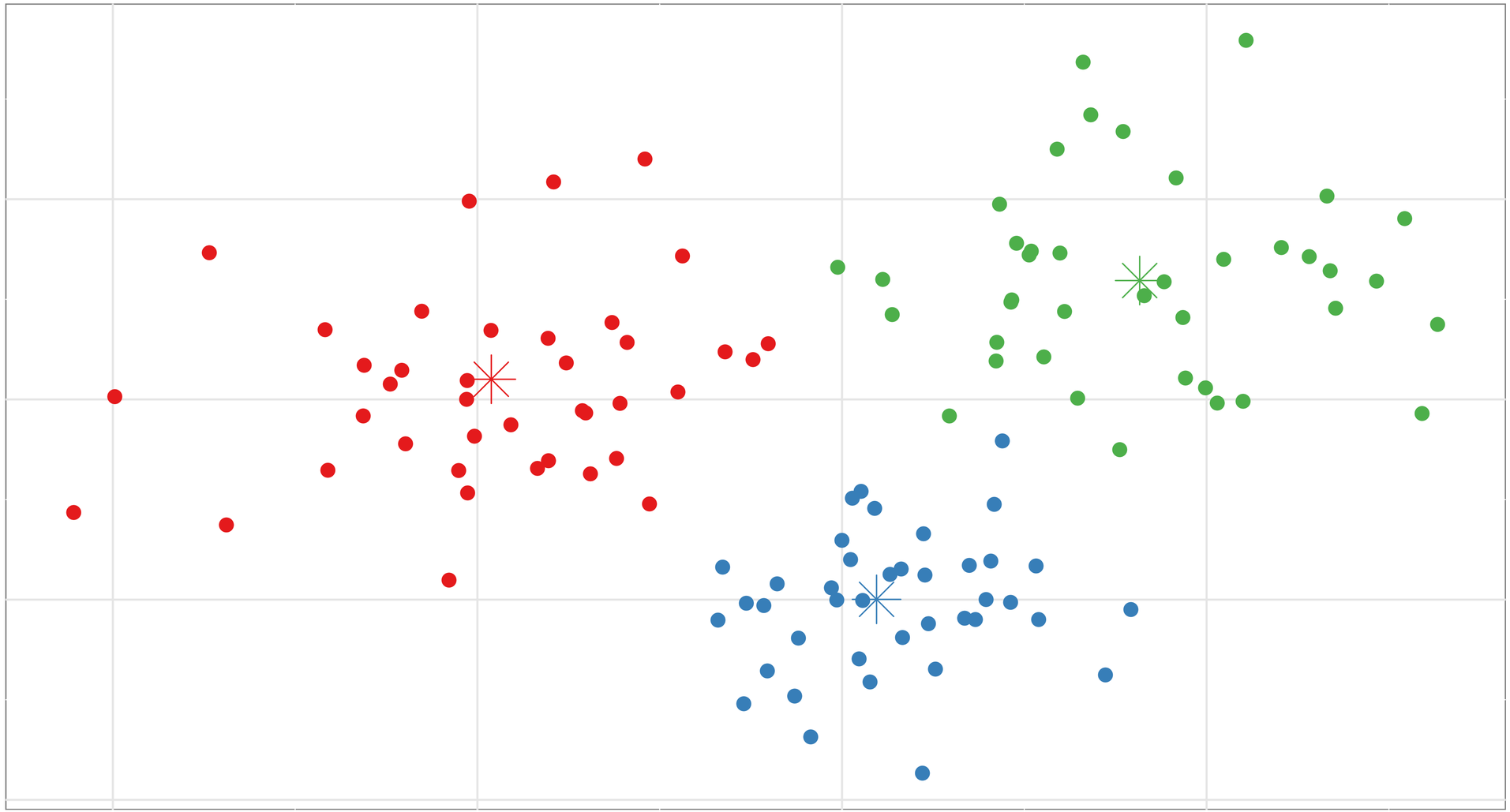
Step 2: Assign each data point to the nearest centroid (3)



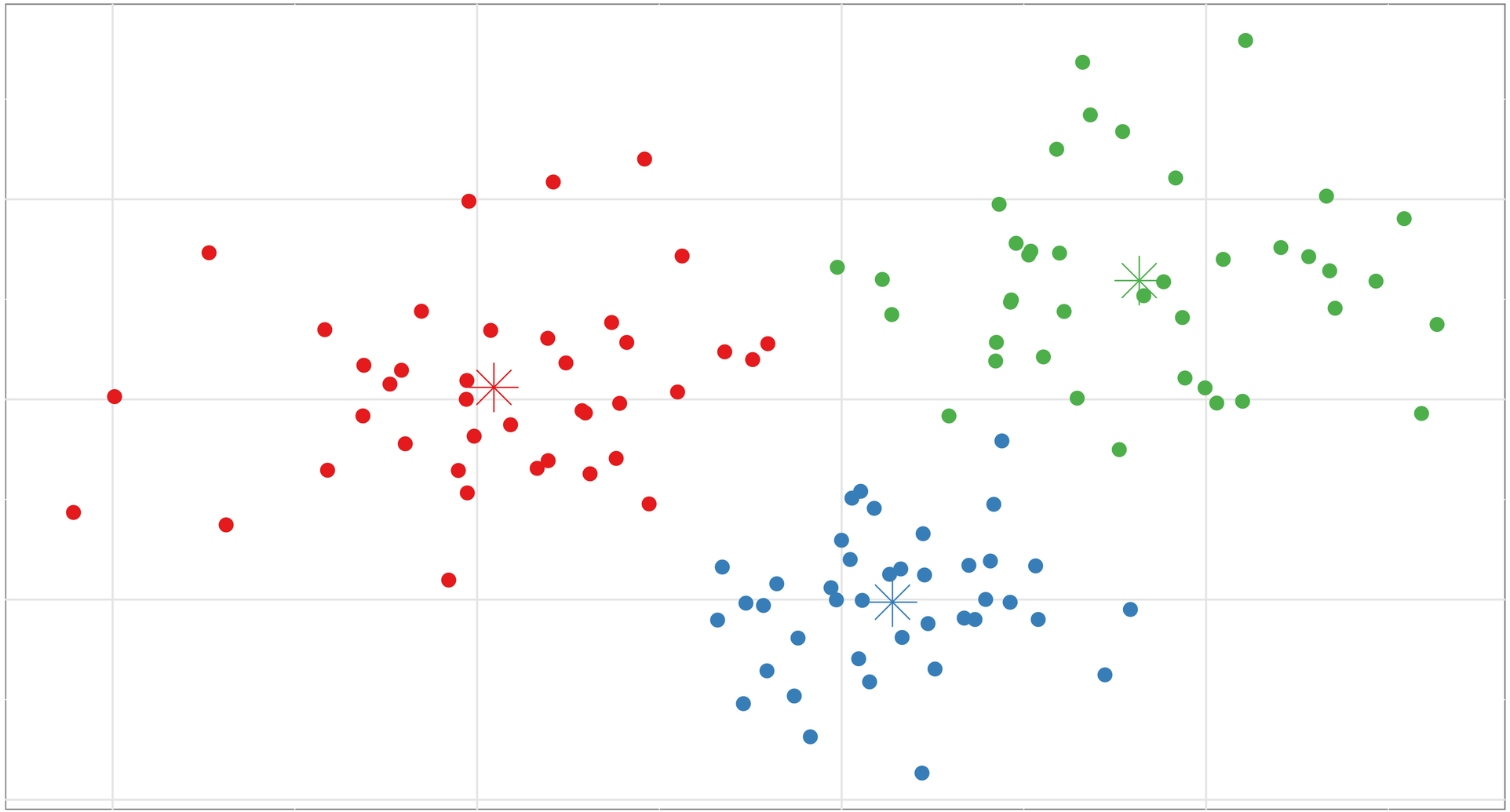
Step 3: Move centroid to center of assigned points (3)



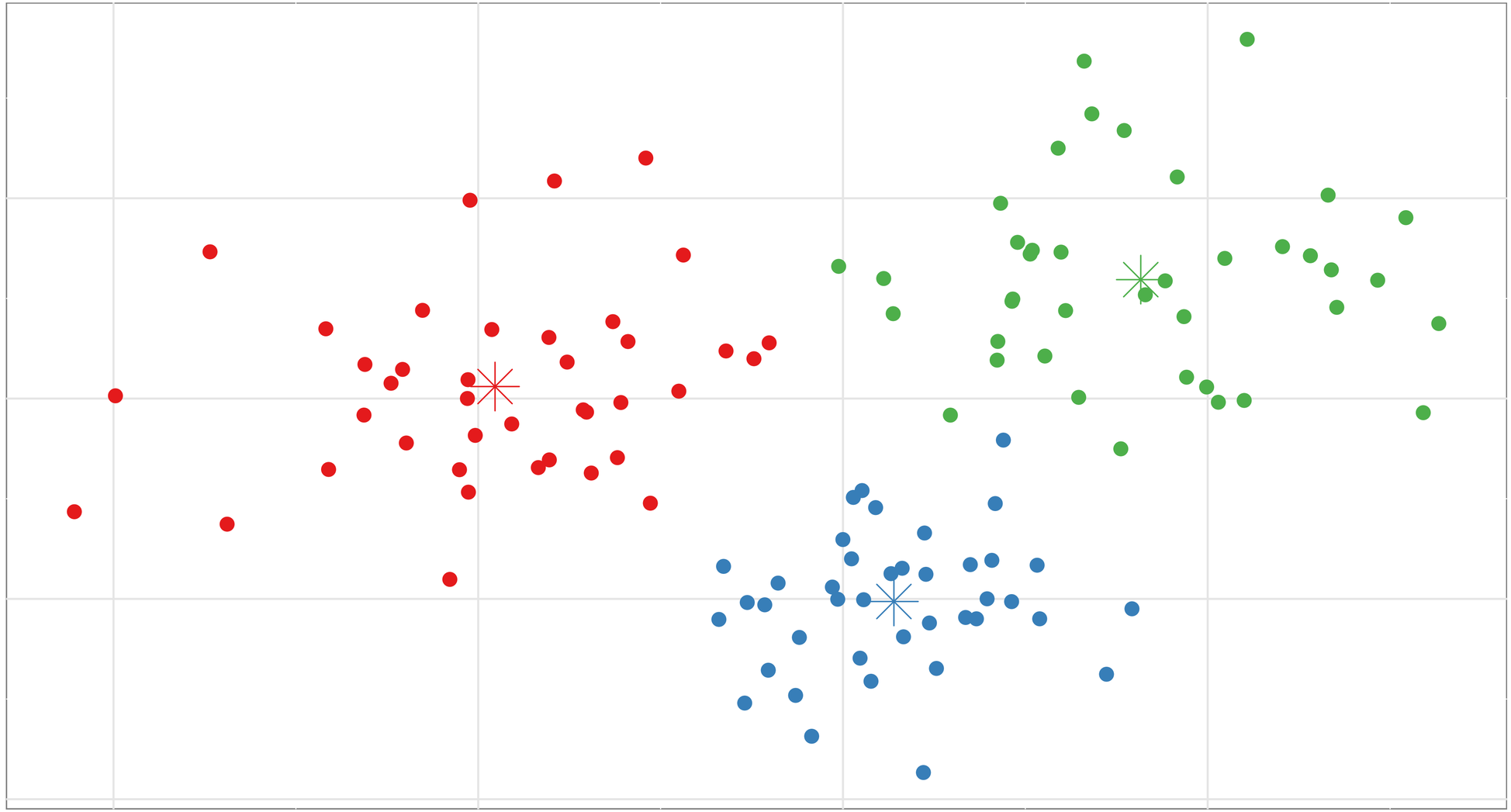
Step 2: Assign each data point to the nearest centroid (4)



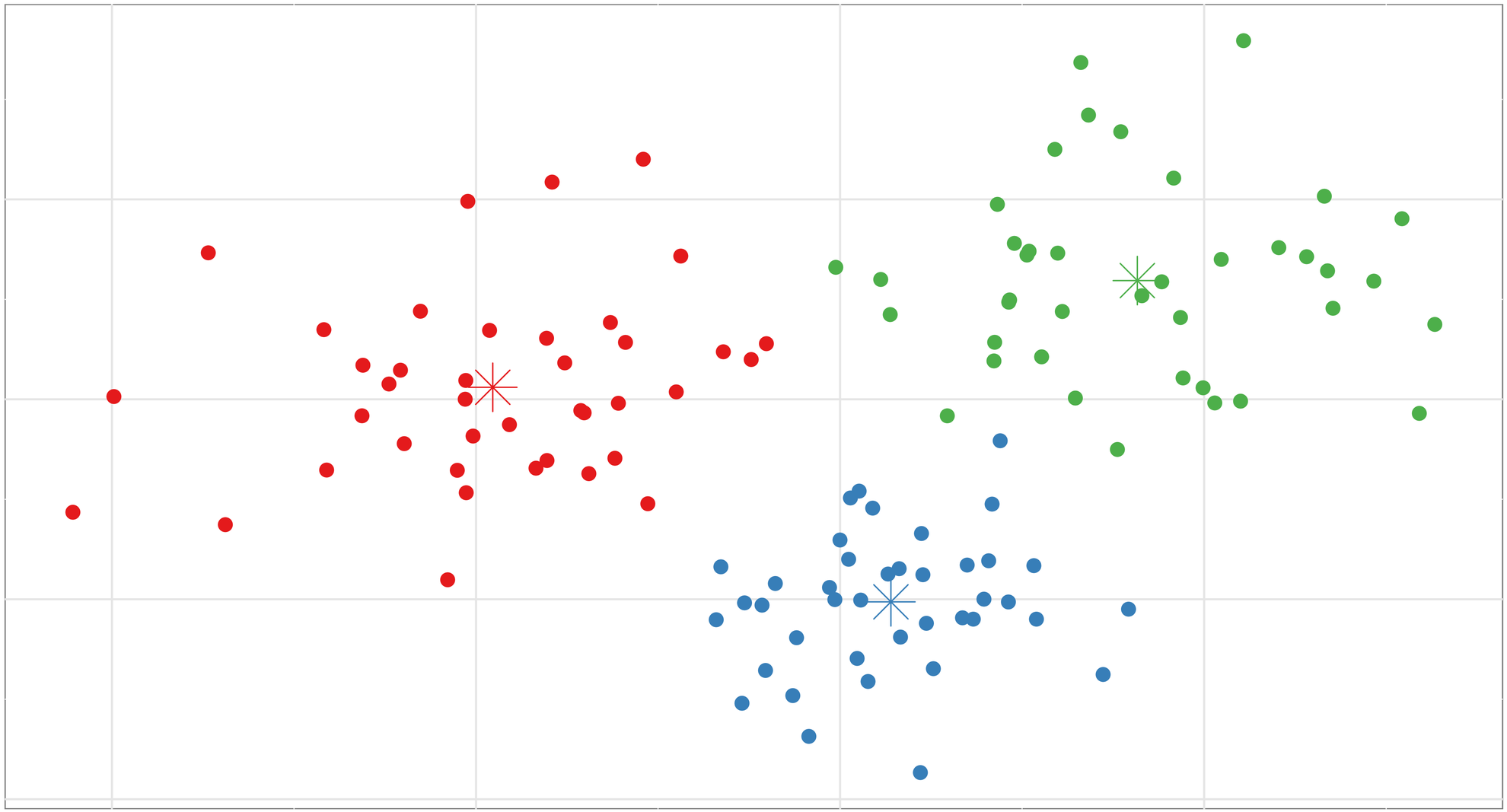
Step 3: Move centroid to center of assigned points (4)



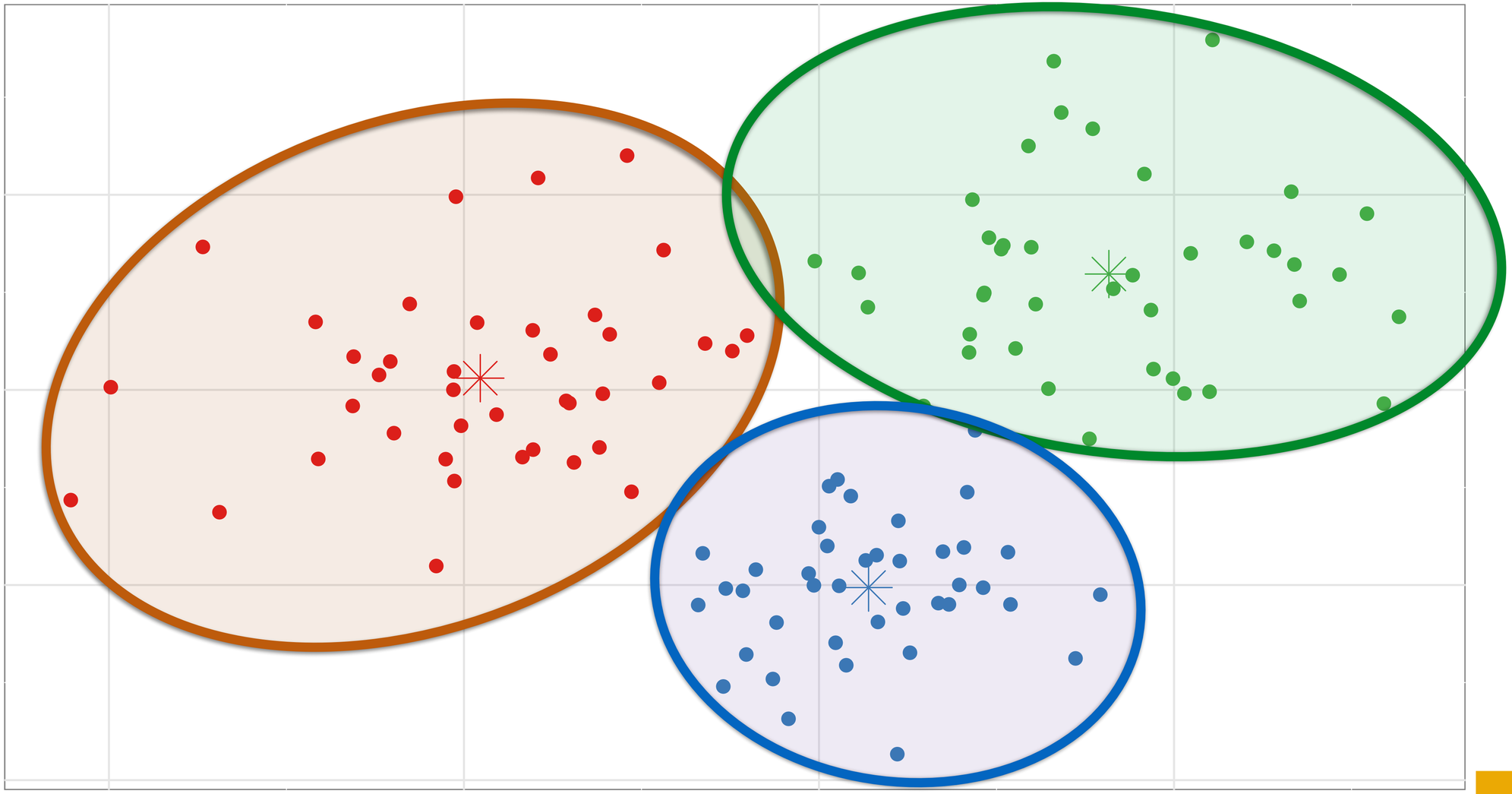
Step 2: Assign each data point to the nearest centroid (5)



Step 3: Move centroid to center of assigned points (5)



Step 3: Move centroid to center of assigned points (5)



K-Means: Got a problem with it?

Before starting, pick the number of clusters, K

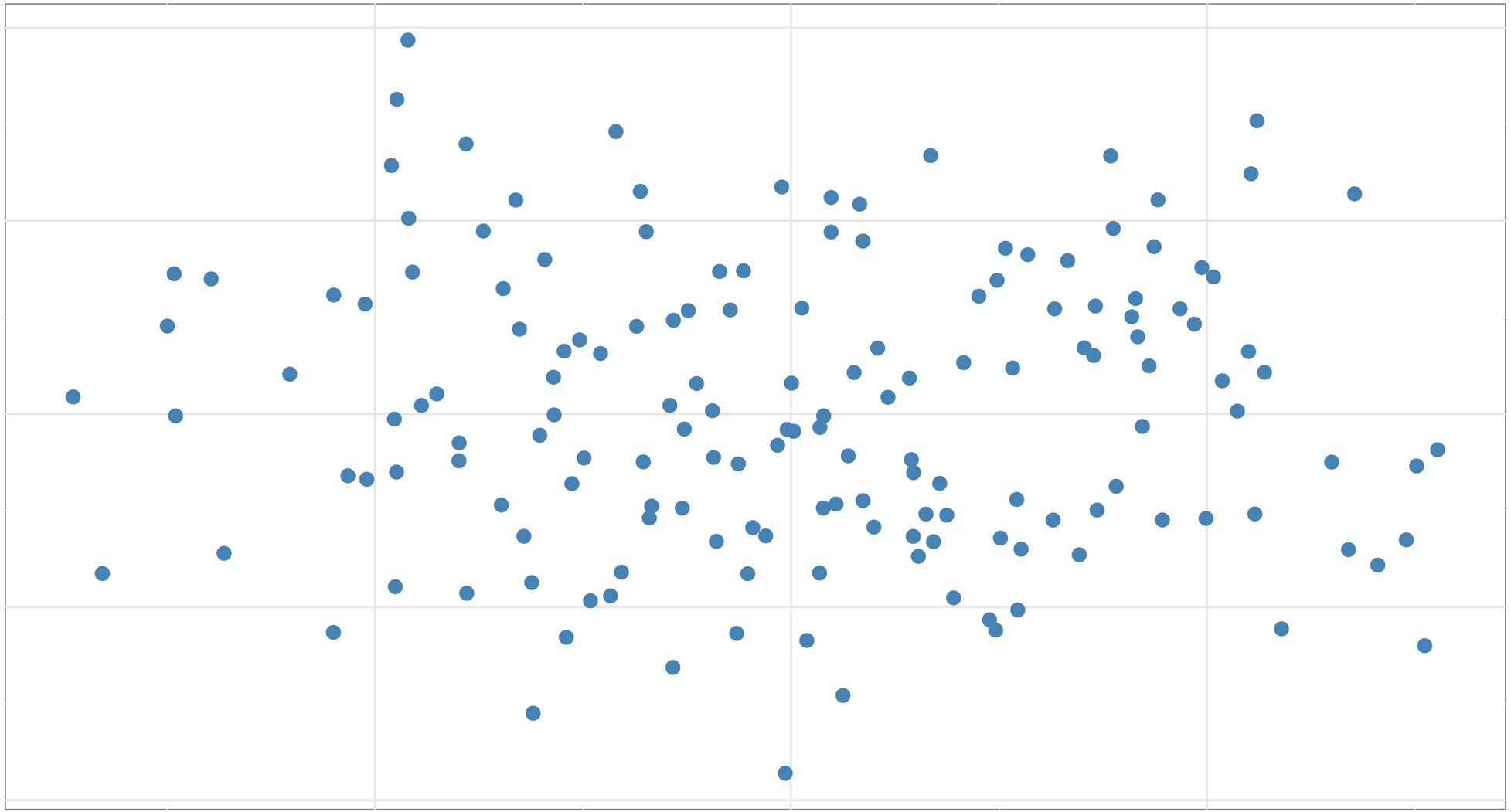
1. Pick K random centroids within data range
2. Assign each data point to the nearest centroid
3. Move centroid to center of assigned points
4. Repeat steps 2 and 3 until centroid stops shifting

K-Means: Got a problem with it?

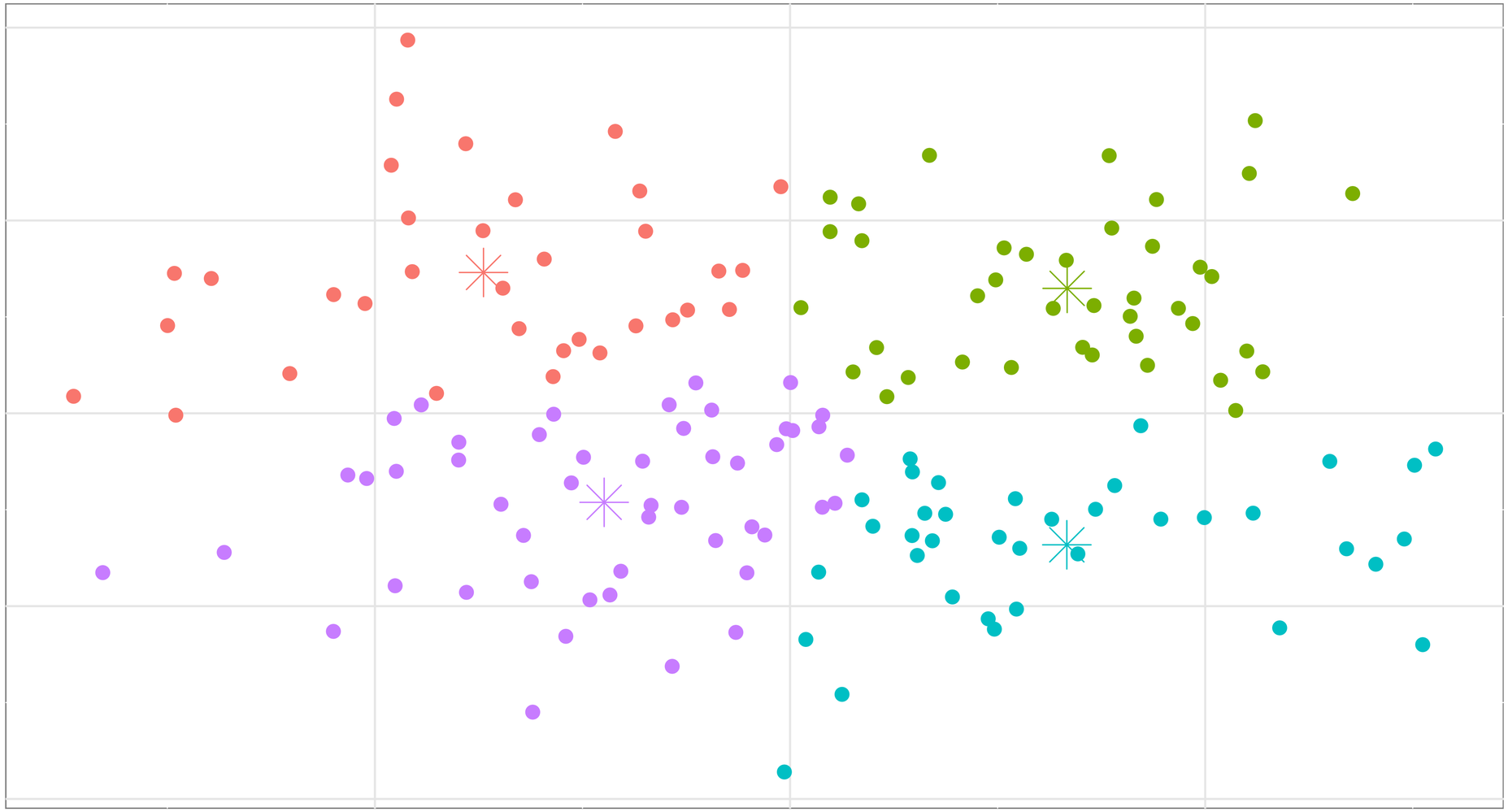
Before starting, pick the number of clusters, K Subjective

1. Pick K random centroids within data range
2. Assign each data point to the nearest centroid
3. Move centroid to center of assigned points Sensitive to Scale
4. Repeat steps 2 and 3 until centroid stops shifting

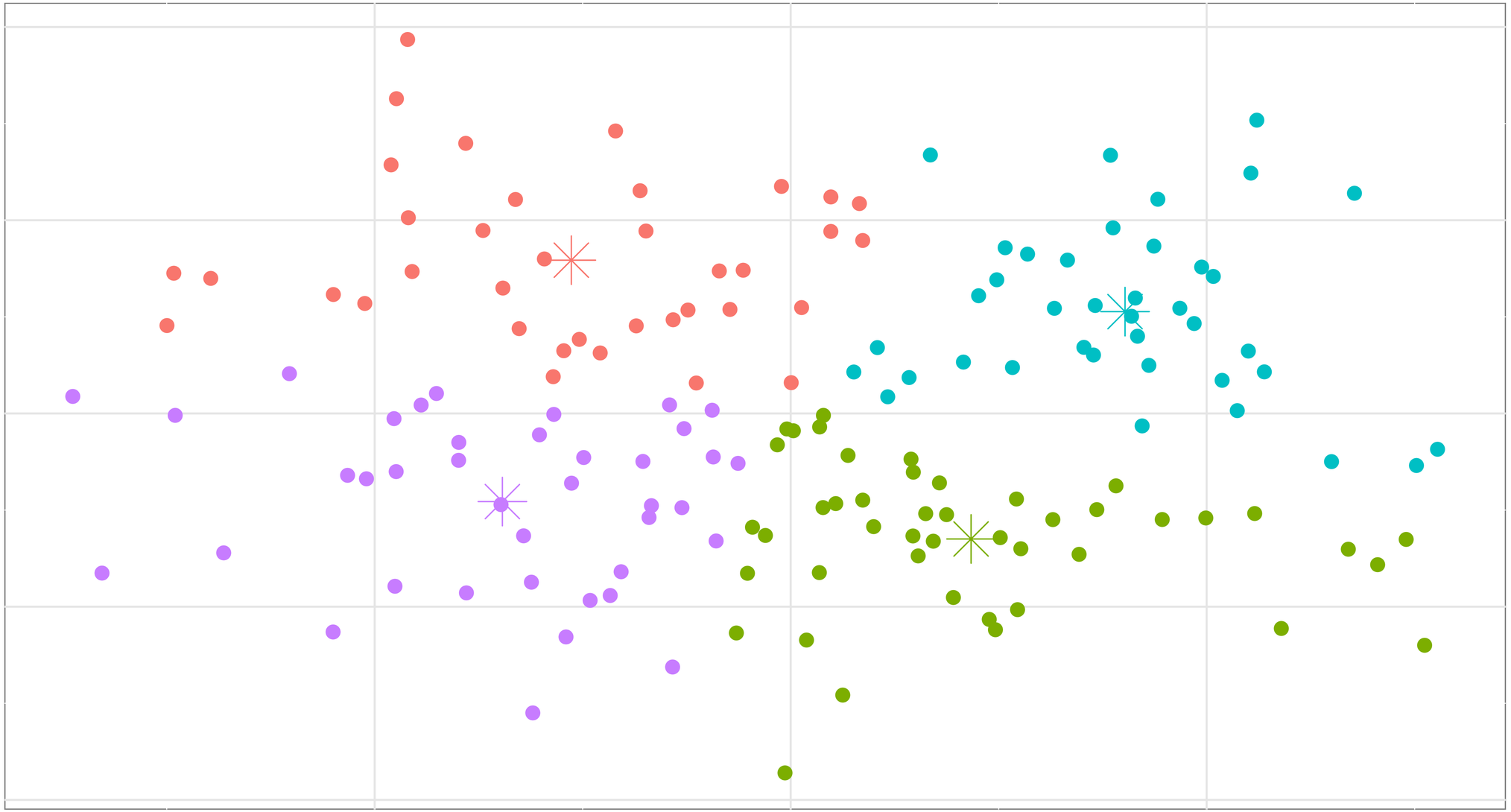
How many clusters?



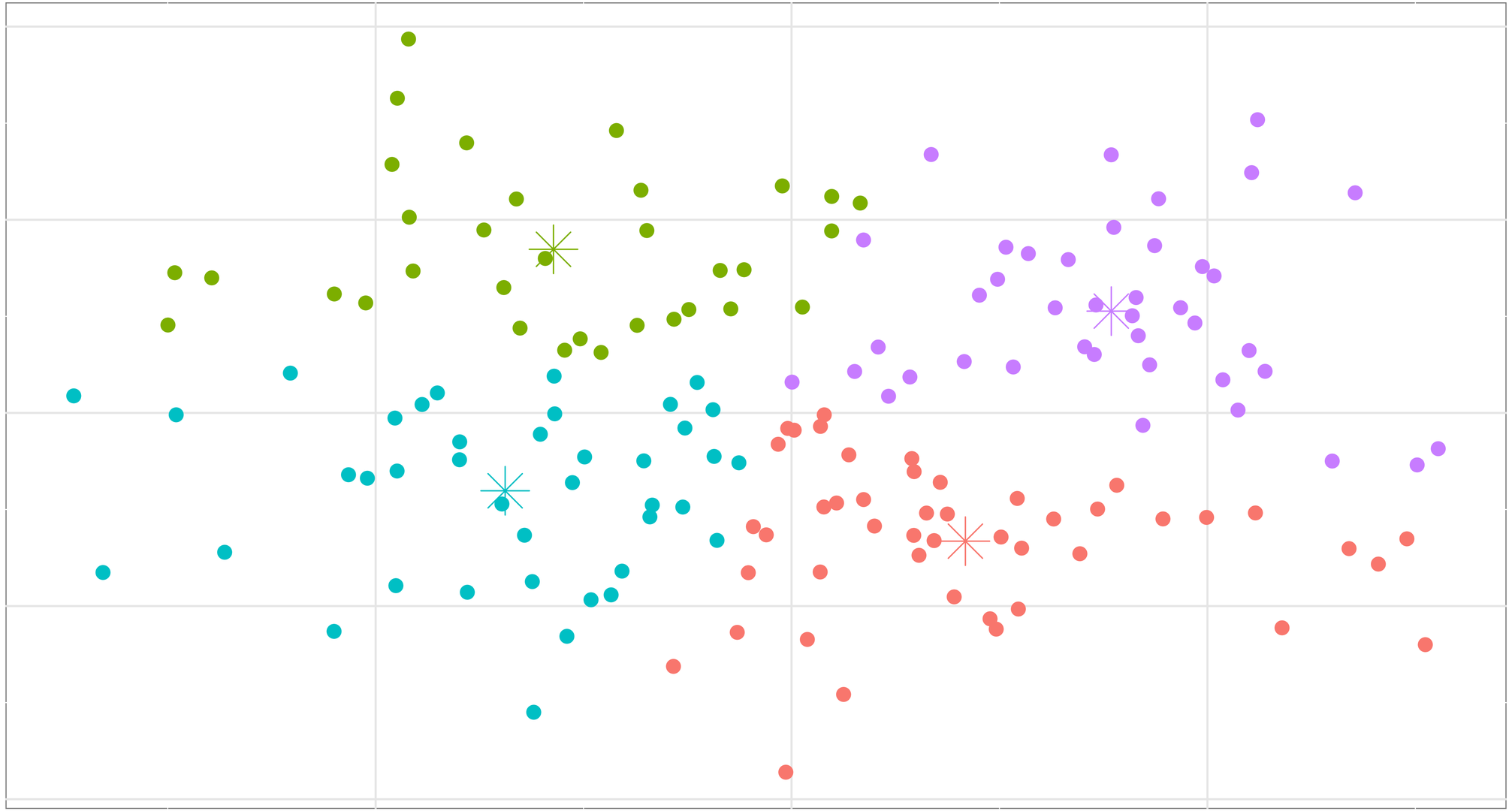
Random Start…



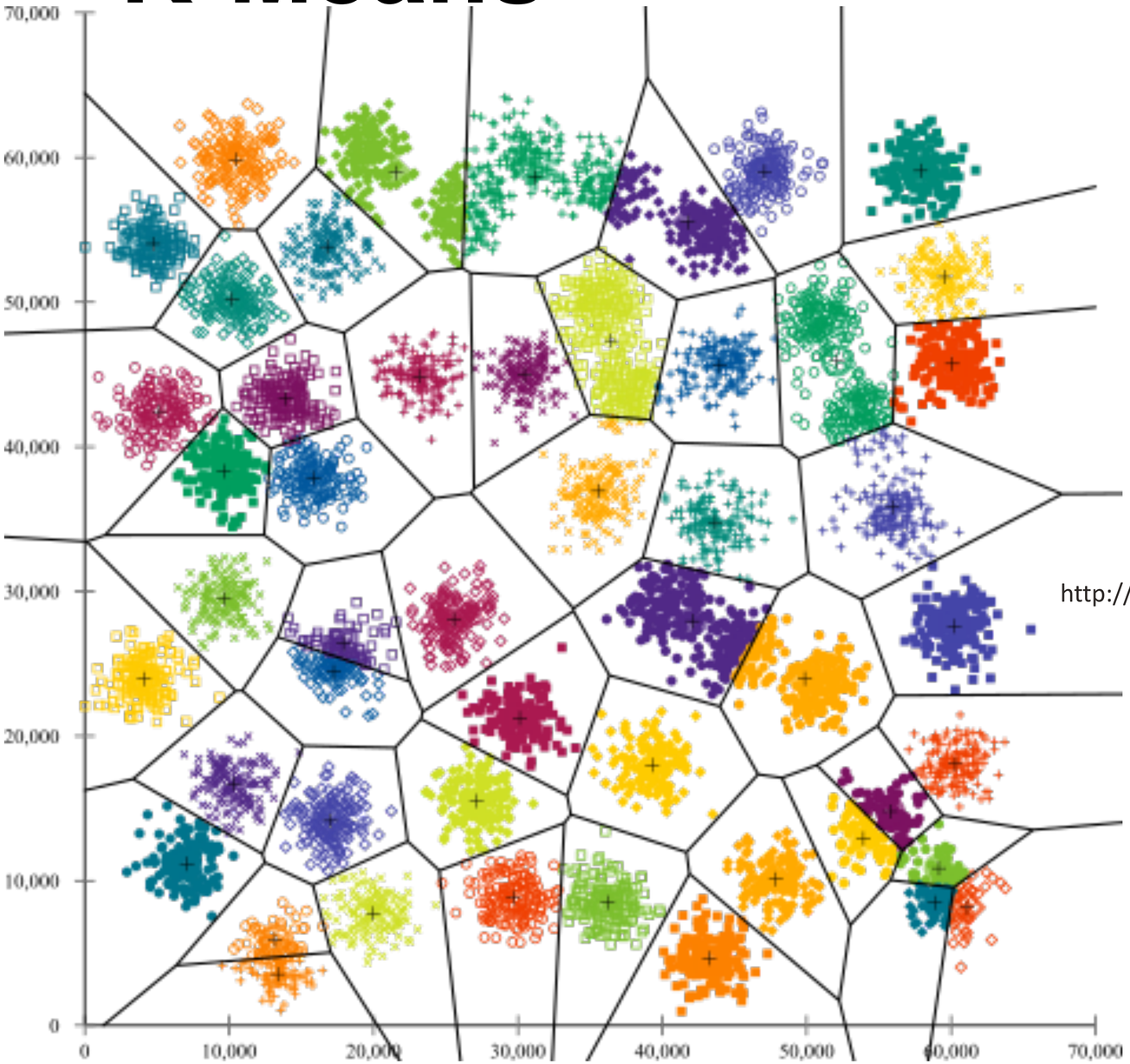
Random Start…



Random Start…



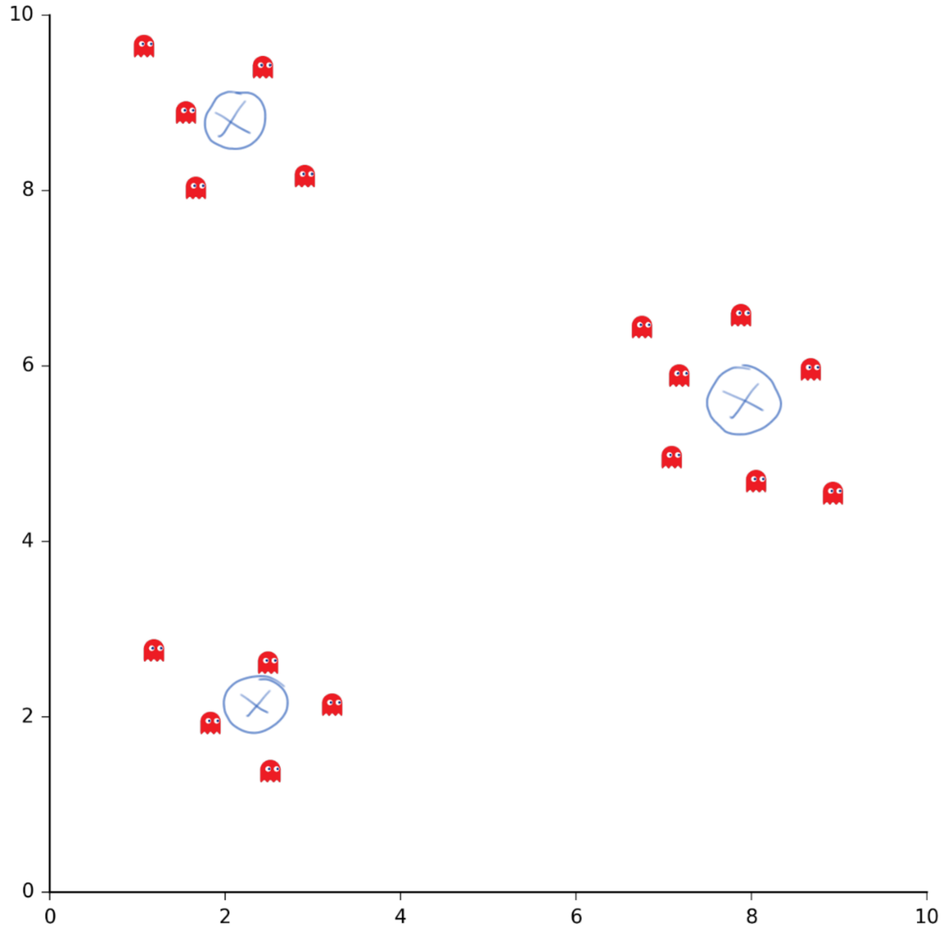
K-Means



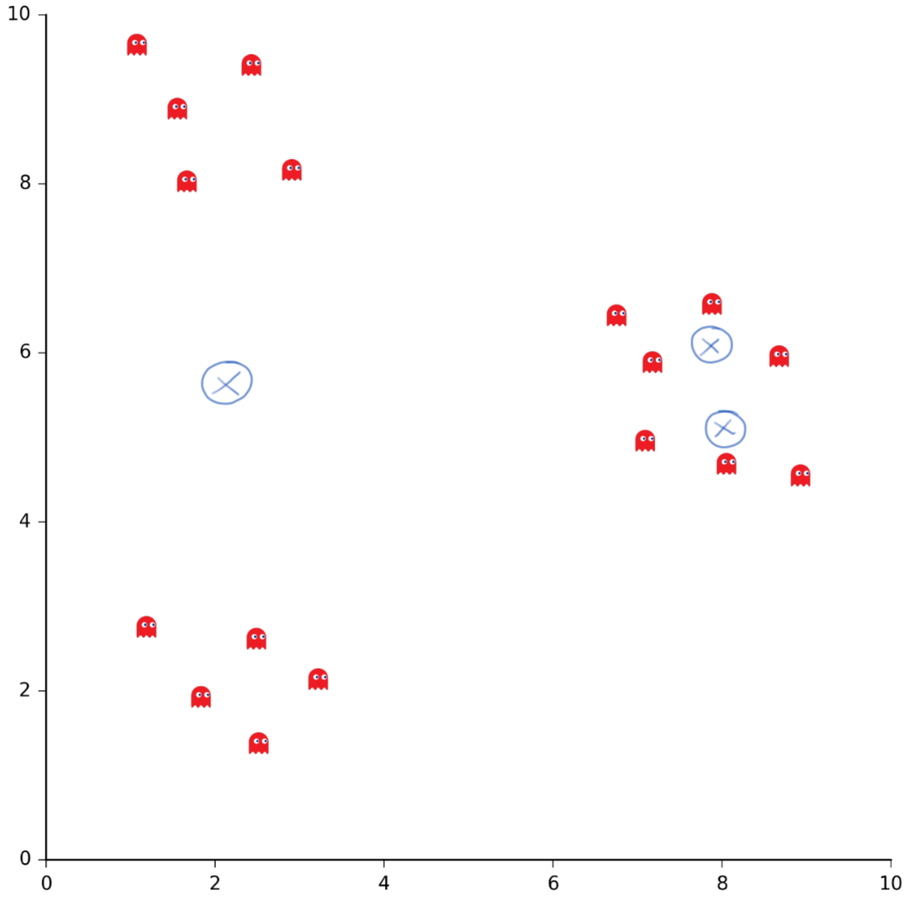
"…it's too easy to throw k-means on your data, and nevertheless get a result out (that is pretty much random, but you won't notice)."
— Anony-Mousse

http://stats.stackexchange.com/questions/133656/how-to-understand-the-drawbacks-of-k-means

Which one is correct?

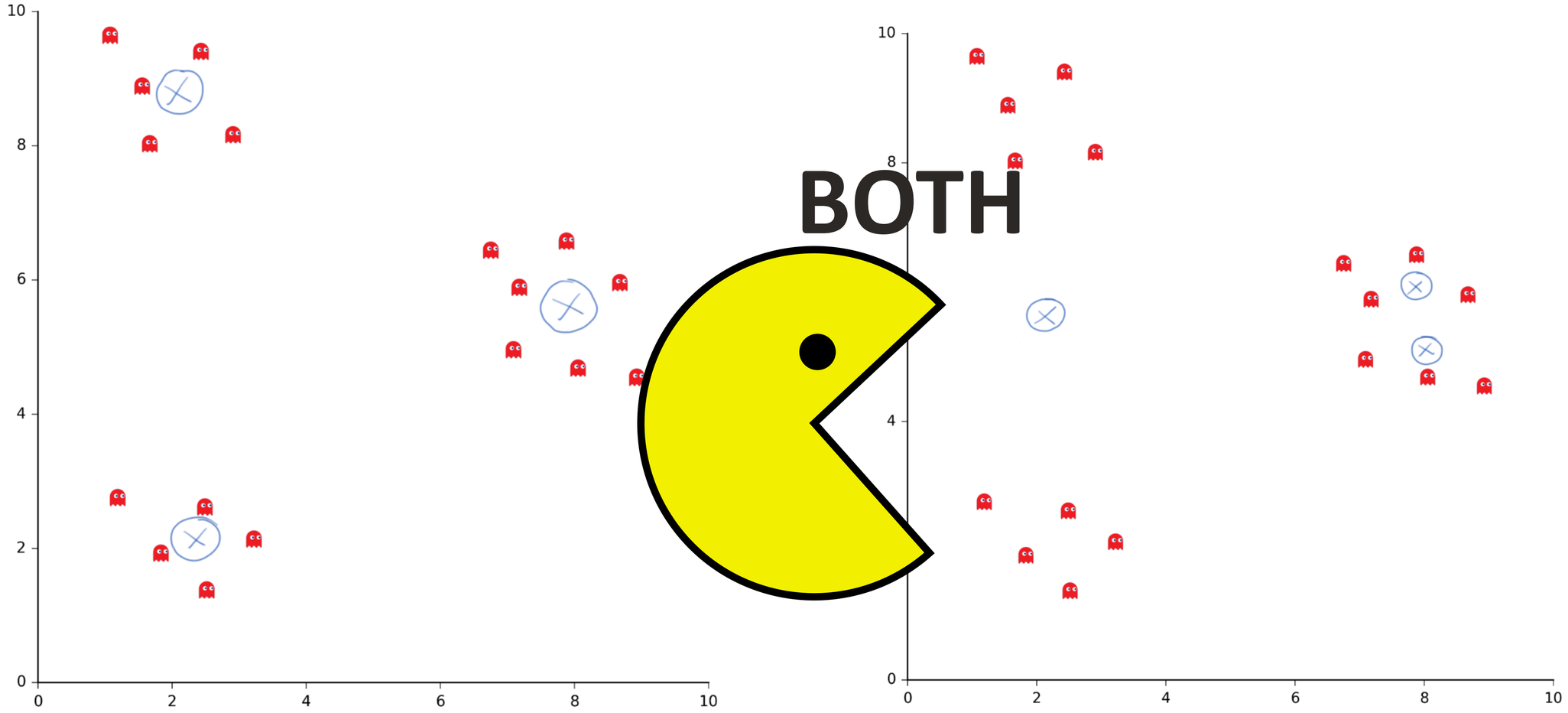


A

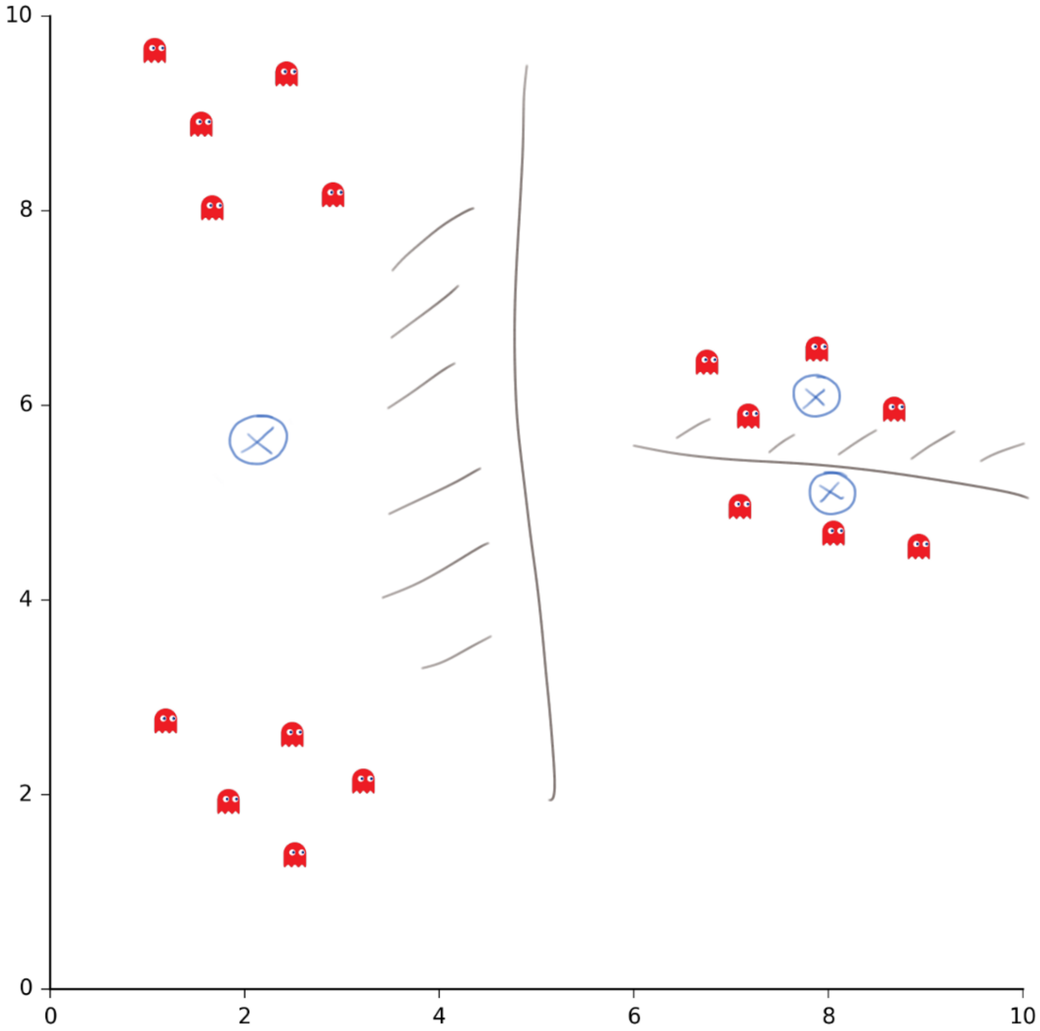


B

Which one is correct?



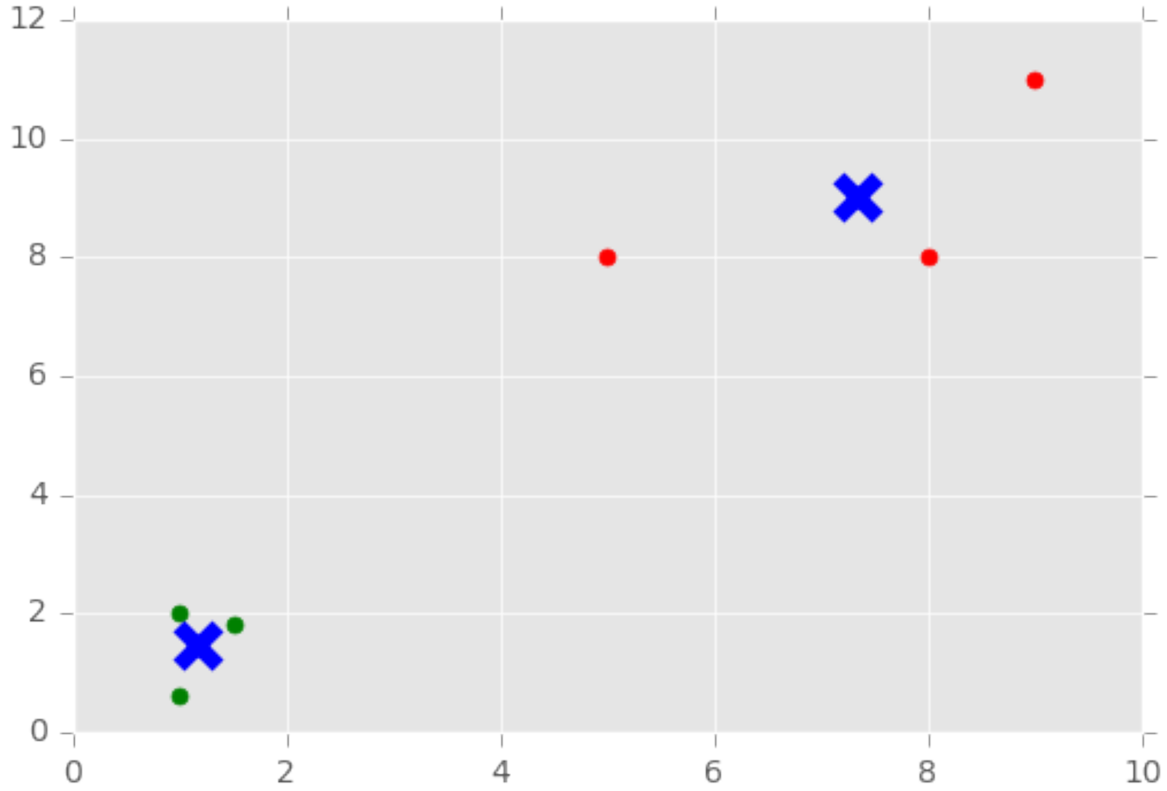
Pain of optimization…Being stuck at sub-optimal local minimum…



Initial guess matters!
Same outcome cannot be guaranteed

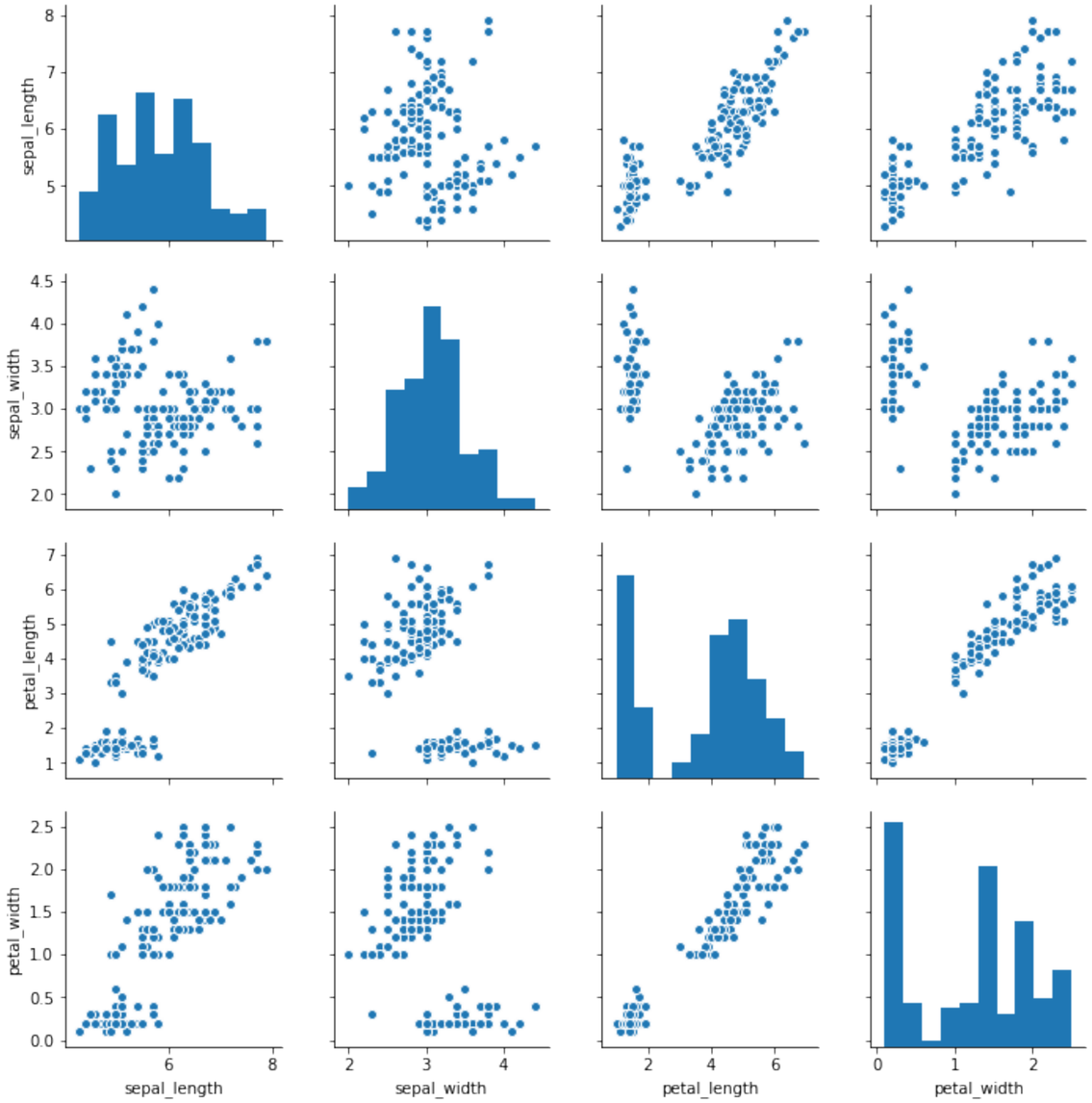
K-Means in practice (Python version)

```
#Import from Scikit-learn
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2)
kmeans.fit(data)

centroids = kmeans.cluster_centers_
labels = kmeans.labels_f
```



The Dataset

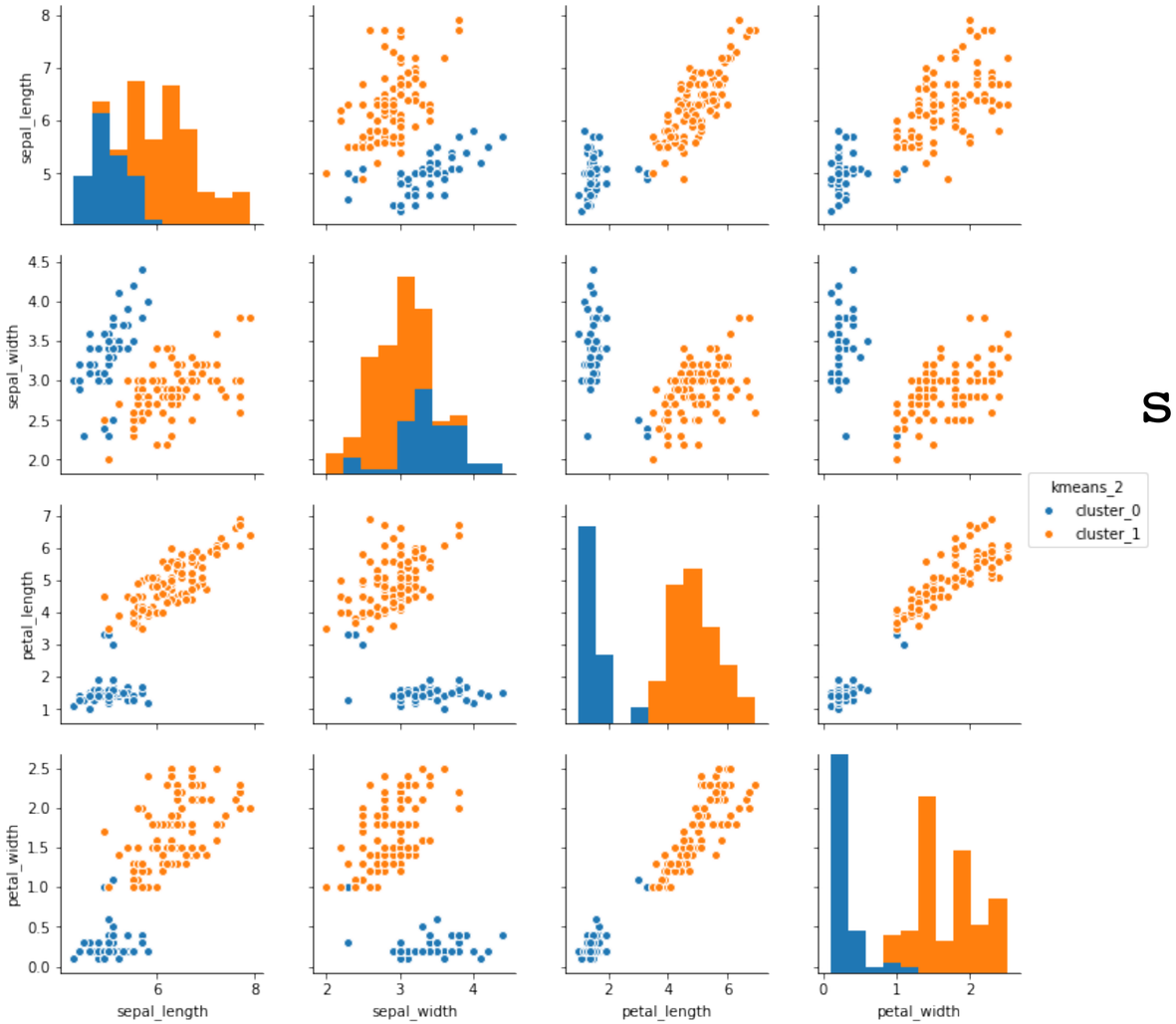
```
sns.pairplot(data)
```



K-Means Clustering

```
kmeans = KMeans(n_clusters=2)
kmeans.fit(data)
```

K-Means Clustering

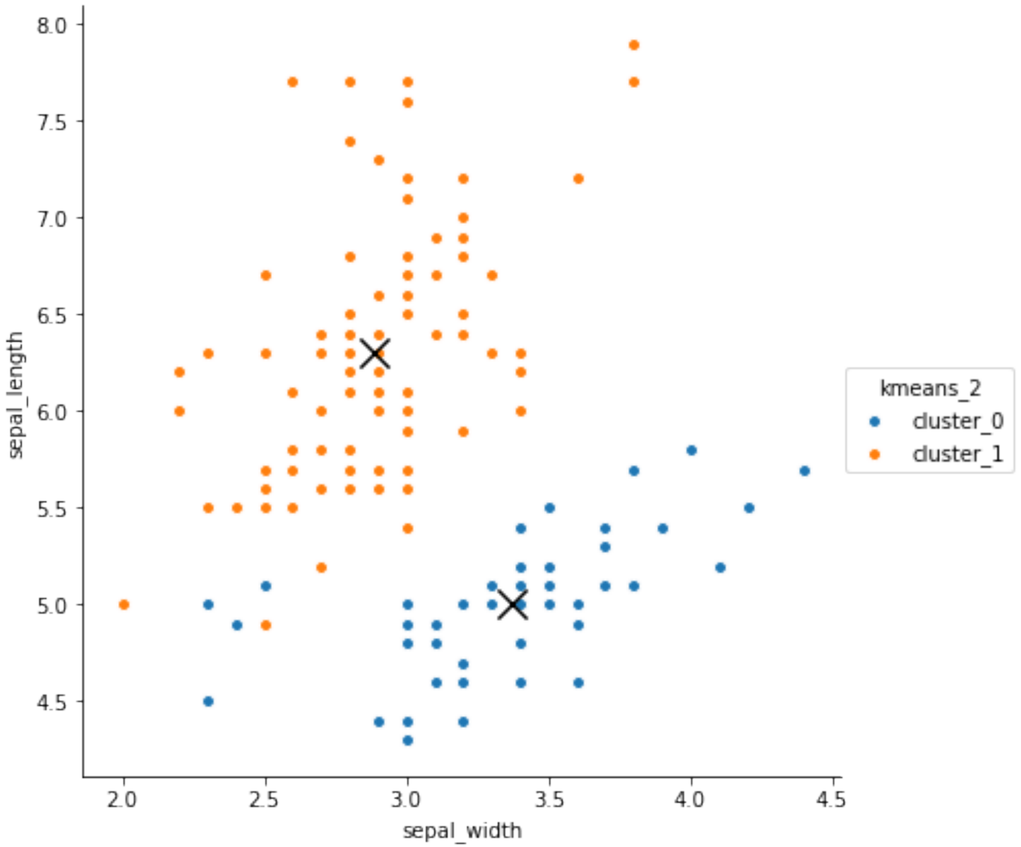


Labels

```
sns.pairplot(data, hue="kmeans_2")
```

K-Means Clustering

```
plt.scatter(<cluster_centers>,<col_2>, linewidths=3, marker='x', s=200,
c='black')
```



K-Means is affected by the scale of every feature.

Feature Scaling

For k-means clustering, features must be scaled to the same ranges of values to contribute "equally" to the euclidean distance calculation.

Each row is transformed per-column by:

- Subtracting from the element in each row the mean for each feature (column) and then taking this value and
- Dividing by that feature's (column's) standard deviation.

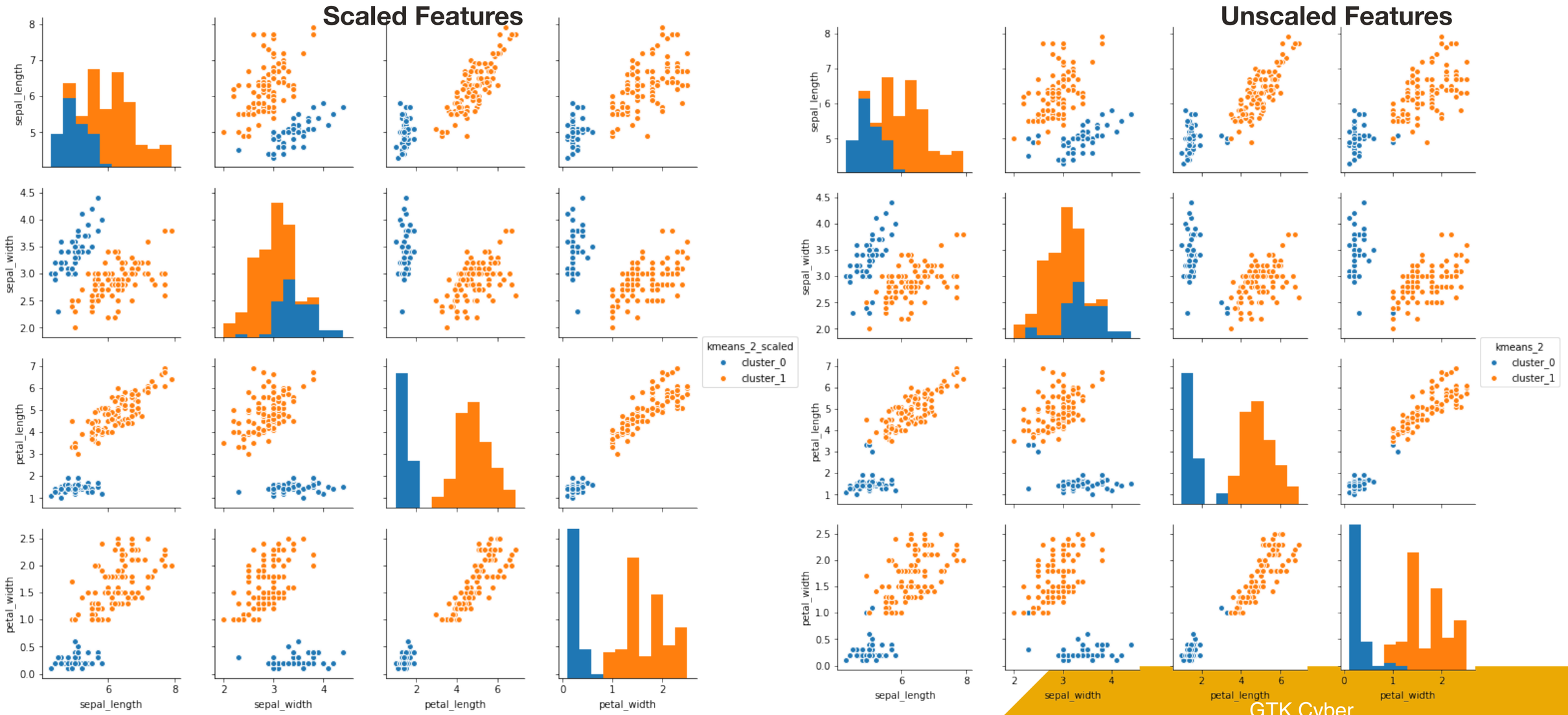
Feature Scaling

```
# center and scale the data
scaler = StandardScaler()

raw_data_scaled = scaler.fit_transform( <data> )

data_scaled = pd.DataFrame( raw_data_scaled,
columns=features )
```

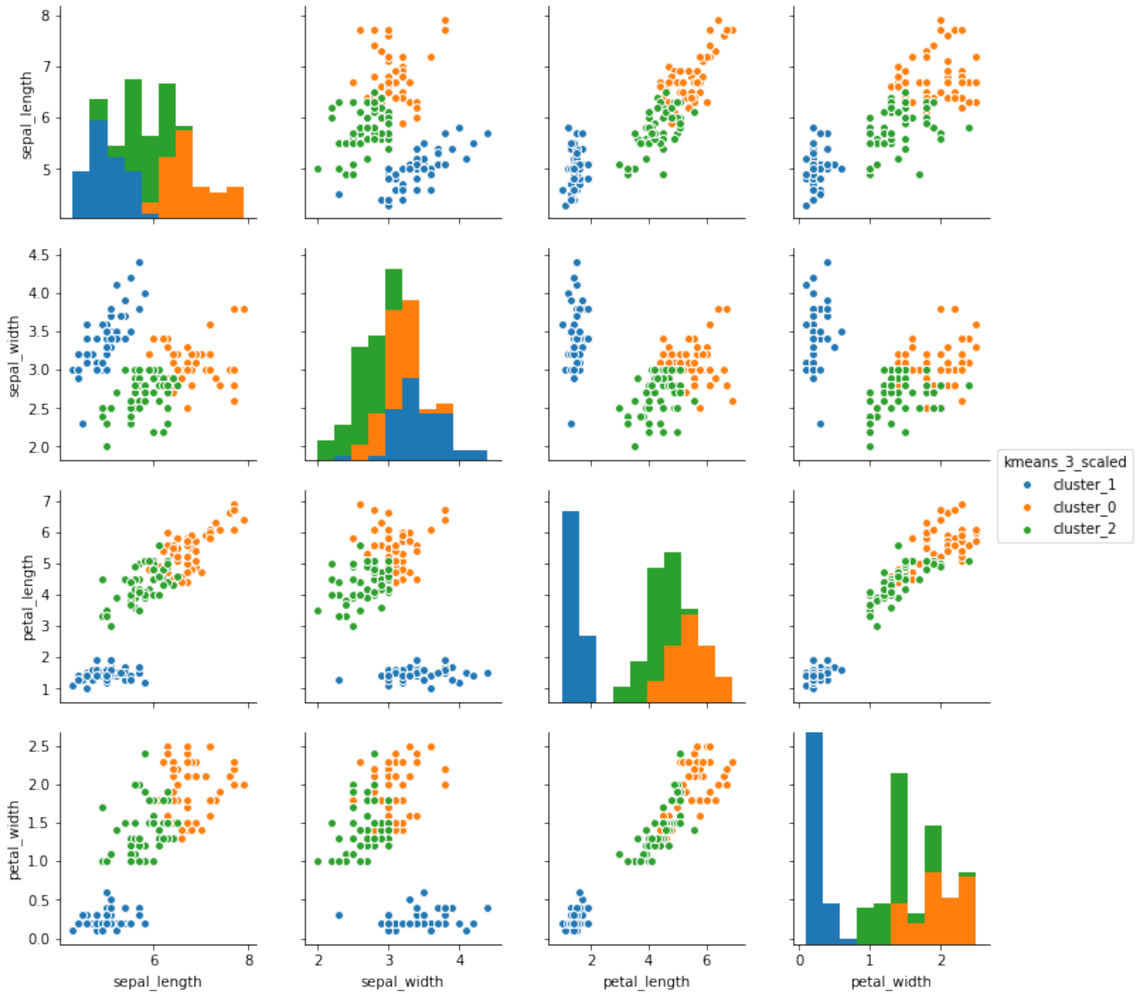
Feature Scaling

```
# K-means on scaled data
km = KMeans( n_clusters=2 )
km.fit( <scaled_data> )
```

Feature Scaling



More Clusters

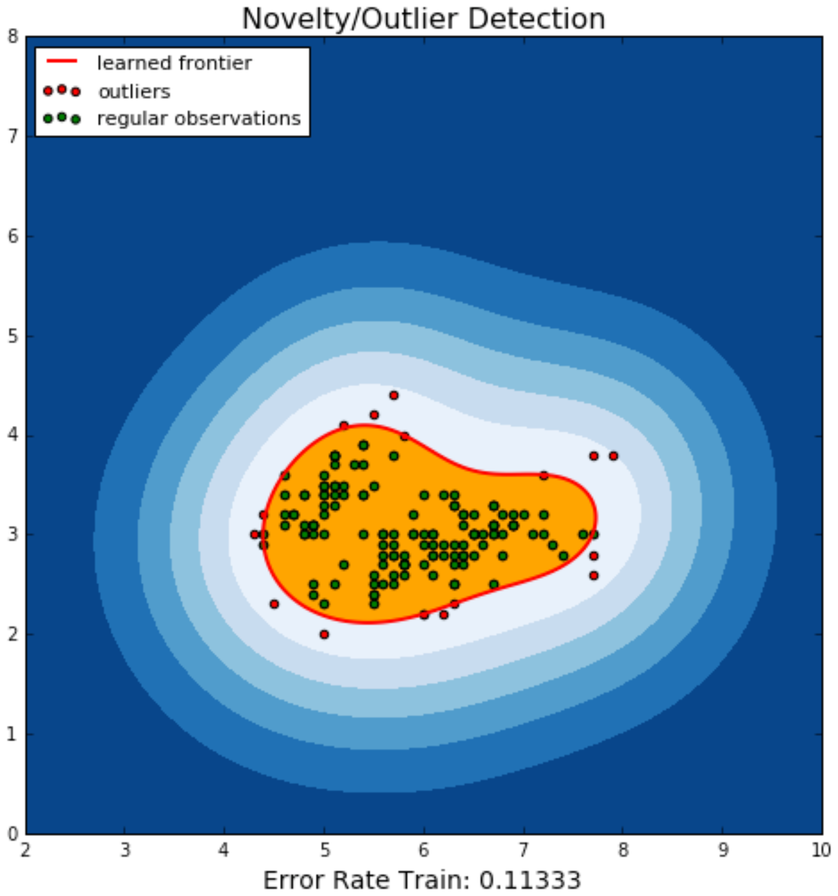



```
km3 = KMeans(n_clusters=3)
km3.fit(scaled_data)
```

Outlier Detection

```
clf = svm.OneClassSVM( tol=0.001, nu=0.1)
clf.fit(X)
target_pred_outliers=clf.predict(X)
```

Delete n% of “outlier data”,
here ~10%



http://scikit-learn.org/stable/modules/generated/sklearn.svm.OneClassSVM.html

Evaluating your Model

The Silhouette Coefficient is a common metric for evaluating clustering "performance" in situations when the "true" cluster assignments are not known.

b = mean distance to next nearest cluster

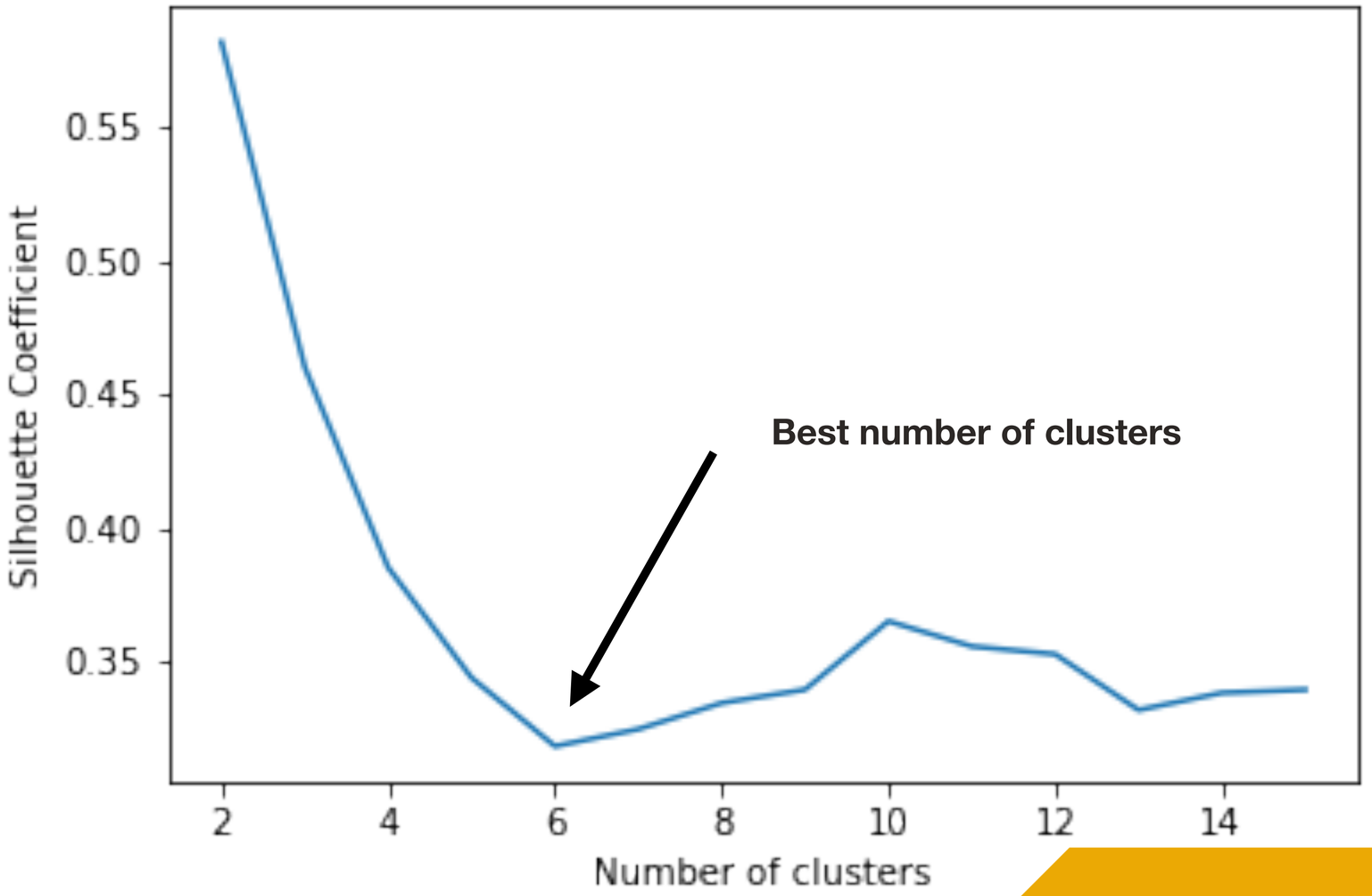
a = mean distance to other points in cluster

```
silhouette_coeff = (b - a) / max(a,b)
```

Evaluating your Model

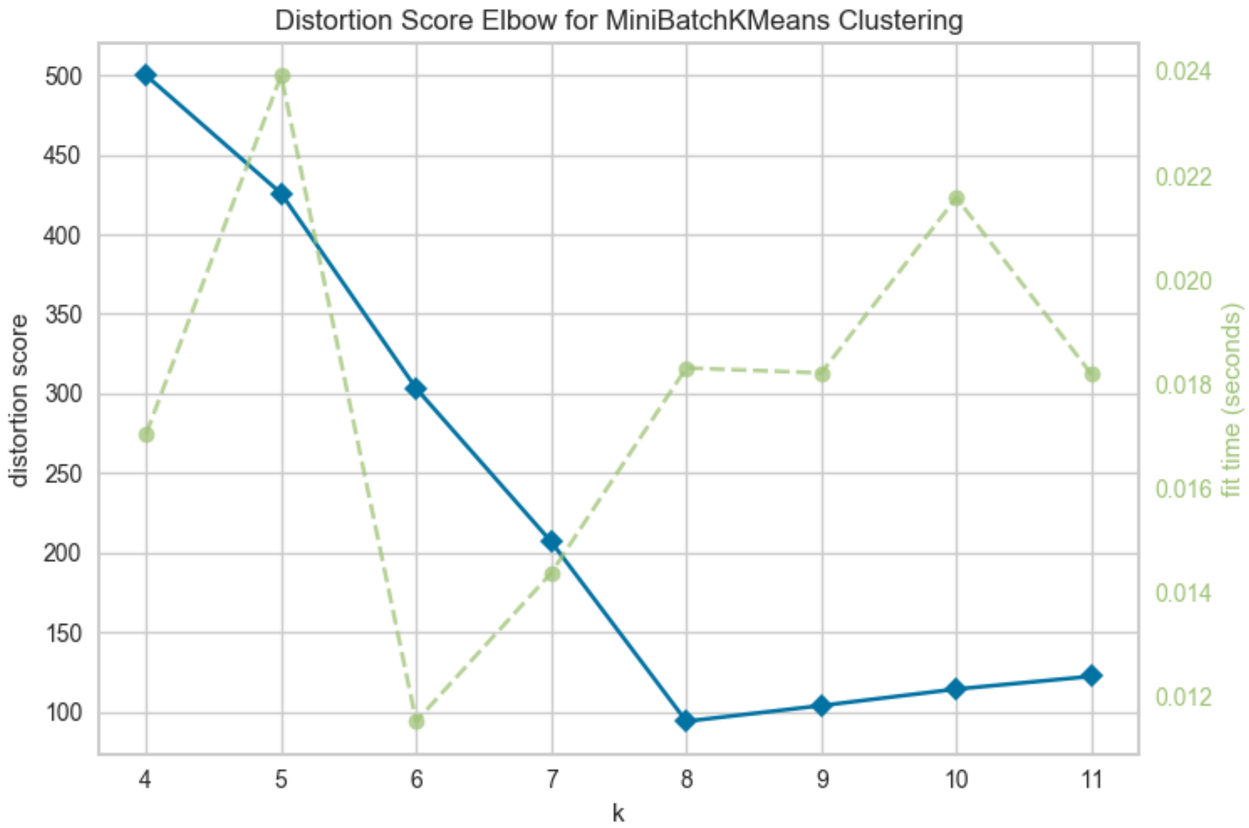
```
k_range = range(2,16)
scores = []
for k in k_range:
    km_ss = KMeans(n_clusters=k, random_state=1)
    km_ss.fit(iris_data_scaled)
    scores.append(silhouette_score(<data>,
km_ss.labels_))
```

Evaluating your Model



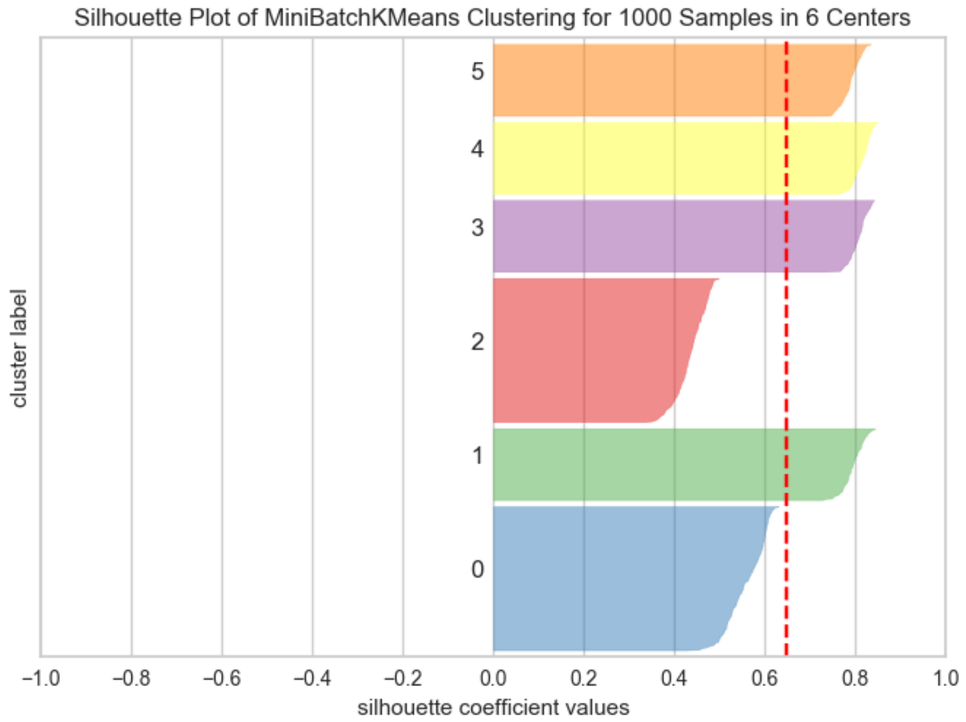
Evaluating your Model

```
from yellowbrick.cluster import KElbowVisualizer
visualizer = KElbowVisualizer(KMeans(), k=(4,12))

visualizer.fit(X)
visualizer.poof()
```



Evaluating your Model

```
from yellowbrick.cluster import SilhouetteVisualizer
model = MiniBatchKMeans(6)
visualizer = SilhouetteVisualizer(model)

visualizer.fit(X)
visualizer.poof()
```



In Class Exercise

Please complete Worksheet 5.1: Clustering

Other uses of unsupervised learning: Dimensionality Reduction

Other uses of unsupervised learning: Dimensionality Reduction

- Dimensionality reduction reduces the number of features in a dataset without losing the information in a data set
- Common technique: **Principal Component Analysis (PCA)**

Other uses of unsupervised learning: Dimensionality Reduction

- PCA attempts to combine the highly correlated features and represent this data with a smaller number of linearly uncorrelated features.
- The algorithm keeps performing this correlation reduction, finding the directions of maximum variance in the original high dimensional data and projecting them onto a smaller dimensional space.
- These newly derived components are known as principal components.

PCA Example

```
from sklearn.decomposition import PCA

n_components = 784
whiten = False

pca = PCA(n_components=n_components, whiten=whiten)

features_train_PCA = pca.fit_transform(features_train_PCA)
features_train_PCA = pd.DataFrame(data= features_train_PCA,
index=train_index)

print("Variance Explained by all 784 principal components: ", \
      sum(pca.explained_variance_ratio_))
```

The variance of all features is 1.0 (everything)

https://github.com/aapatel09/handson-unsupervised-learning/blob/master/03_dimensionality_reduction.ipynb

PCA Example

```
Variance Captured by First 10 Principal Components:  [0.48876238]
Variance Captured by First 20 Principal Components:  [0.64398025]
Variance Captured by First 50 Principal Components:  [0.8248609]
Variance Captured by First 100 Principal Components:  [0.91465857]
Variance Captured by First 200 Principal Components:  [0.96650076]
Variance Captured by First 300 Principal Components:  [0.9862489]
```

Bottom line: 98% of the information is contained in 300 features. You can therefore, reduce the size of your dataset by half, theoretically without significantly reducing performance.

https://github.com/aapatel09/handson-unsupervised-learning/blob/master/03_dimensionality_reduction.ipynb

Questions?